AI VIETNAM
All-in-One Course
(TA Session)

MLflow

Extra Class: MLOps



Nguyen-Thuan Duong – TA

Kha-Vi Nguyen – STA

Year 2024

code&data

Outline

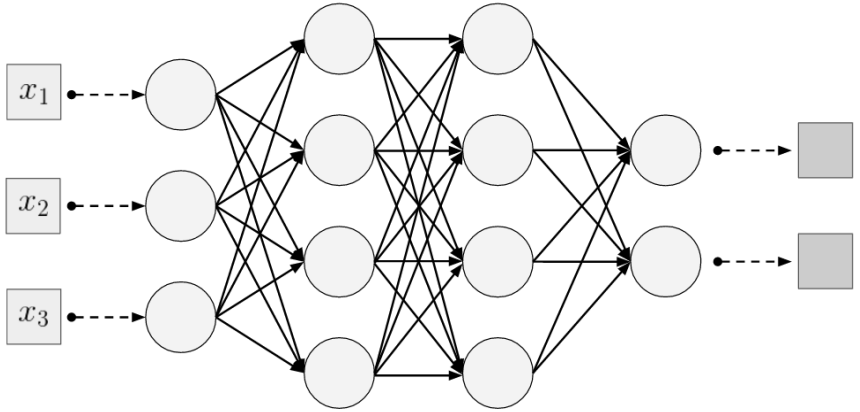
- Problems
- Introduction
- MLflow
- Pipeline
- Practices
- Question

Problems

Problems

❖ Getting Started

Model Architecture



Hyper-parameters

Batch_size = 32
Learning_rate = 1e-4

Weight

Weight Optimization

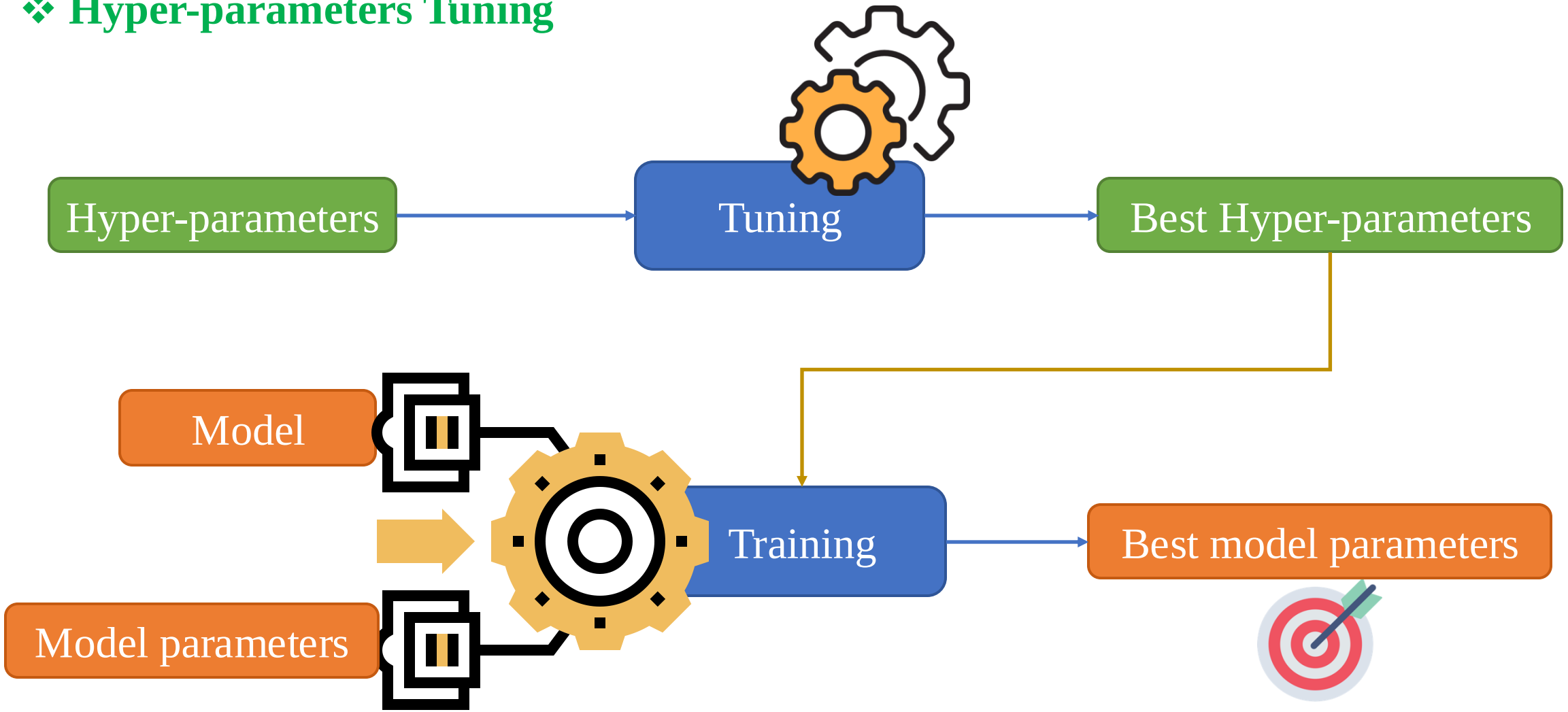
Optimizer



0.72

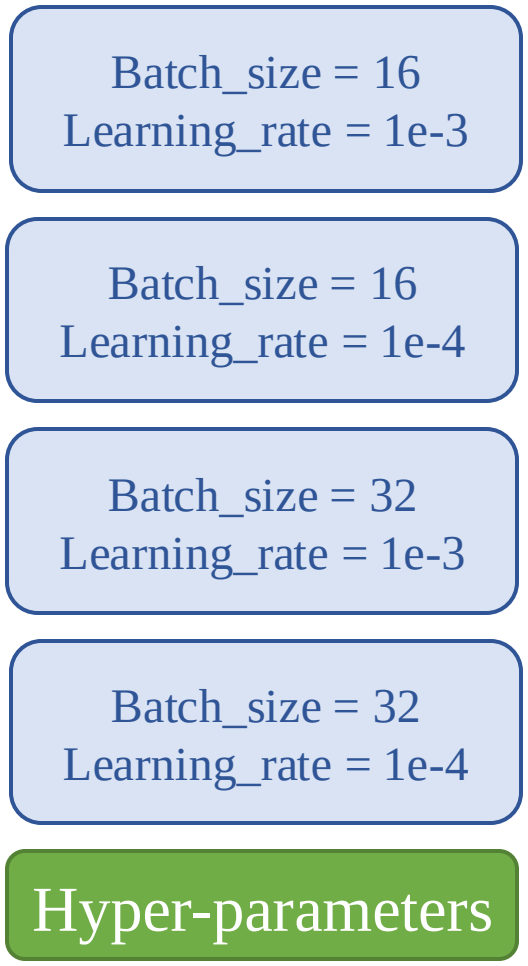
Problems

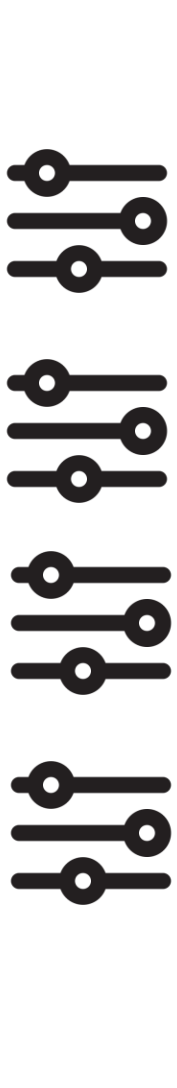
❖ Hyper-parameters Tuning

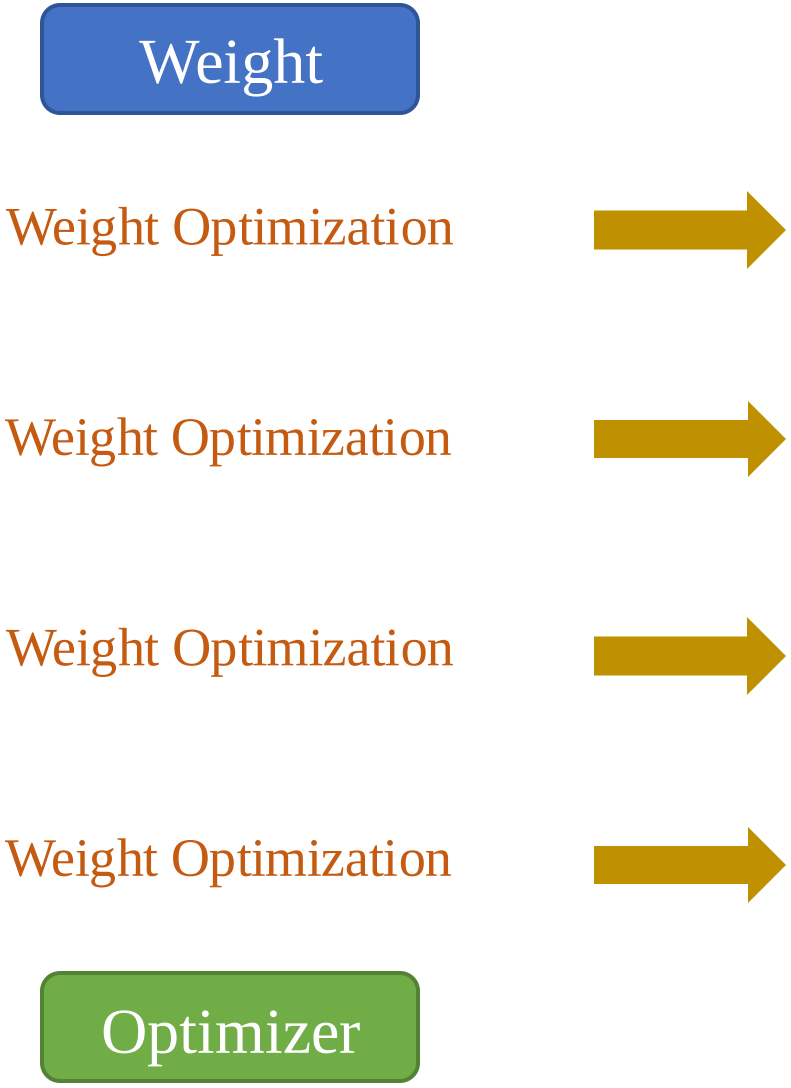


Problems

❖ Hyper-parameters Tuning





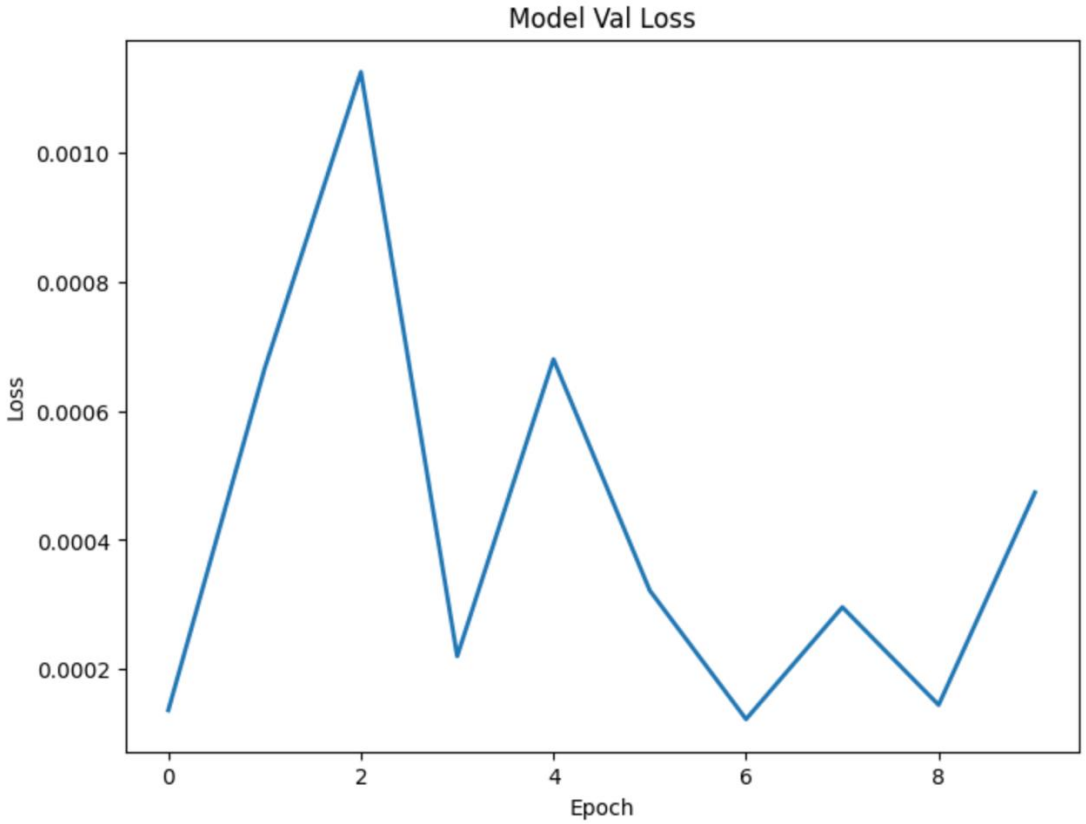




Problems

❖ Training and logging

```
Epoch 1, train loss: 0.005840, val loss: 0.000136
Epoch 2, train loss: 0.000341, val loss: 0.000666
Epoch 3, train loss: 0.000371, val loss: 0.001125
Epoch 4, train loss: 0.000362, val loss: 0.000220
Epoch 5, train loss: 0.000325, val loss: 0.000680
Epoch 6, train loss: 0.000318, val loss: 0.000322
Epoch 7, train loss: 0.000413, val loss: 0.000122
Epoch 8, train loss: 0.000364, val loss: 0.000296
Epoch 9, train loss: 0.000363, val loss: 0.000144
Epoch 10, train loss: 0.000290, val loss: 0.000474
```



RUN 1

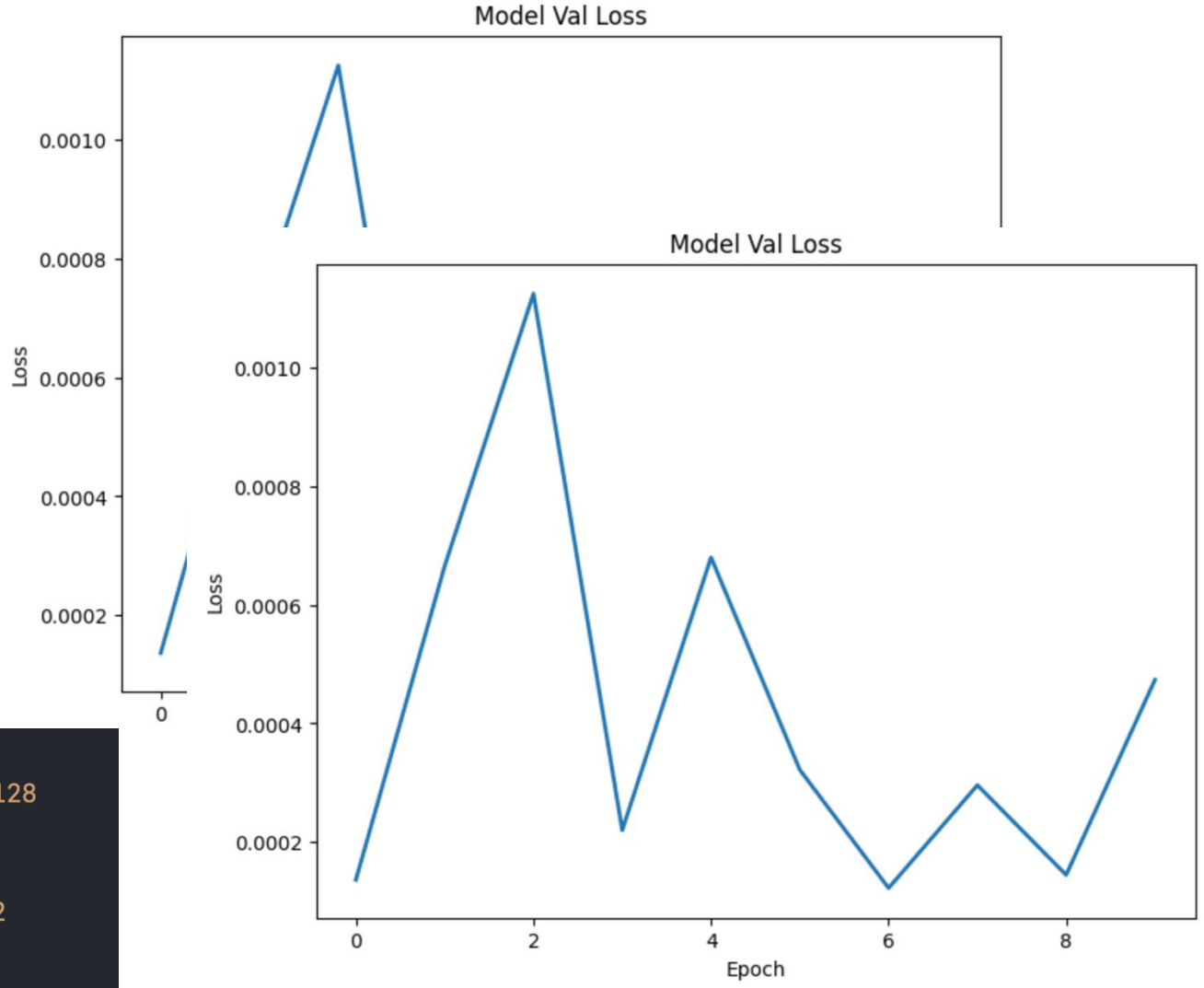
```yaml
version: v0.1
hidden_size: 128
num_layers: 2
lr: 0.001
batch_size: 16
epochs: 30
lookback: 14
device: cpu
```

Problems

❖ Training and logging

```
Epoch 1, train loss: 0.005840, va
Epoch 2, train loss: 0.000341, va
Epoch 3, train loss: 0.000371, va
Epoch 4, train loss: 0.000362, va
Epoch 5, train loss: 0.000325, va
Epoch 6, train loss: 0.000318, va
Epoch 7, train loss: 0.000413, va
Epoch 8, train loss: 0.000364, val loss: 0.000296
Epoch 9, train loss: 0.000363, val loss: 0.000144
Epoch 10, train loss: 0.000290, val loss: 0.000474
```

```yaml
version: v0.1
hidden_size: 128
num_layers: 2
lr: 0.001
batch_size: 16
epochs: 30
lookback: 14
device: cpu
```




```
Epoch 1, train loss: 0.005840, val loss: 0.000136
Epoch 2, train loss: 0.000341, val loss: 0.000666
Epoch 3, train loss: 0.000371, val loss: 0.001125
Epoch 4, train loss: 0.000362, val loss: 0.000220
Epoch 5, train loss: 0.000325, val loss: 0.000680
Epoch 6, train loss: 0.000318, val loss: 0.000322
Epoch 7, train loss: 0.000413, val loss: 0.000122
Epoch 8, train loss: 0.000364, val
Epoch 9, train loss: 0.000363, val
Epoch 10, train loss: 0.000290, val
```

```yaml
version: v0.1
hidden_size: 128
num_layers: 2
lr: 0.001
batch_size: 32
epochs: 30
lookback: 14
device: cpu
```

RUN 1, 2

Problems

❖ Training and logging

```
version: v0.1
hidden_size: 128
num_layers: 2
lr: 0.001
batch_size: 16
epochs: 30
```

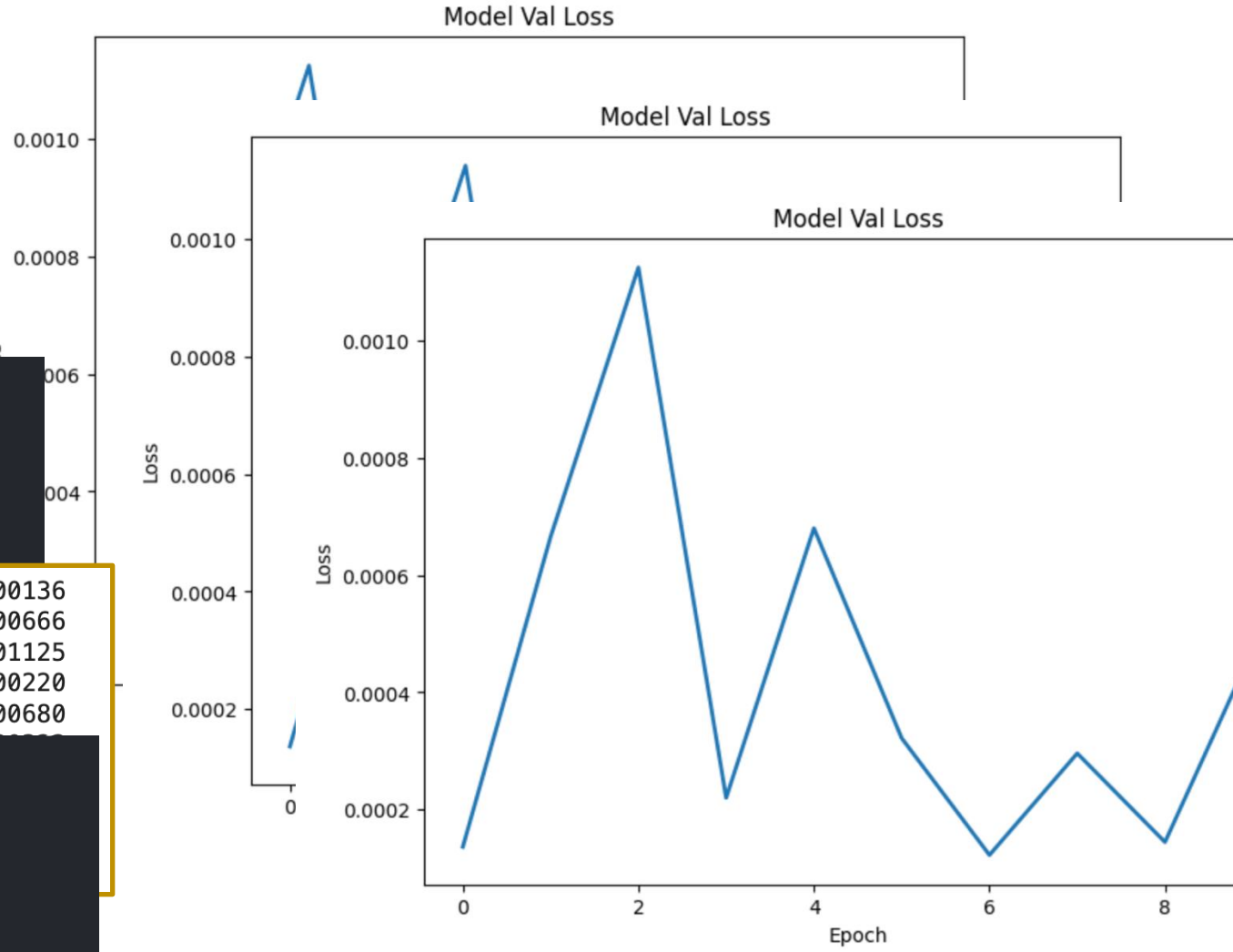
```
Epoch 1, train loss: 0.005840,
Epoch 2, train loss: 0.000341,
Epoch 3, train loss: 0.000371,
Epoch 4, train loss: 0.000362,
Epoch 5, train loss: 0.000325,
Ep
Ep
Ep
Ep
Ep
```

```
Epoch 1, train loss: 0.005840, val loss: 0.000136
Epoch 2, train loss: 0.000341, val
Epoch 3, train loss: 0.000371, val
Epoch 4, train loss: 0.000362, val
Epoch 5, train loss: 0.000325, val
Epoch 6, train loss: 0.000318, val
Epoch 7, train loss: 0.000413, val
Epoch 8, trai
Epoch 9, trai
Epoch 10, tra
```

```
version: v0.1
hidden_size: 128
num_layers: 2
lr: 0.001
batch_size: 32
```

```
Epoch 1, train loss: 0.005840, val loss: 0.000136
Epoch 2, train loss: 0.000341, val loss: 0.000666
Epoch 3, train loss: 0.000371, val loss: 0.001125
Epoch 4, train loss: 0.000362, val loss: 0.000220
Epoch 5, train loss: 0.000325, val loss: 0.000680
Epoch 6, train loss: 0.
Epoch 7, train loss: 0.
Epoch 8, train loss: 0.
Epoch 9, train loss: 0.
Epoch 10, train loss: 0
```

```
version: v0.1
hidden_size: 128
num_layers: 2
lr: 0.001
batch_size: 32
epochs: 50
lookback: 14
device: cpu
```



RUN 1, 2, 3

Problems

❖ Hyperparameter tuning strategies

Grid Search

brute-force method

$$Total\ runs = \prod_{i=1}^{n} |H_i|$$

Random Search

Total runs = Number of random samples

Search with KFold

Total runs * Number of Folds

- n is number of hyperparameters
- $|H_i|$ is number of value of i-th hyperparameters

Introduction

Introduction

❖ Getting Started



Work individually



Team with team

Experimental

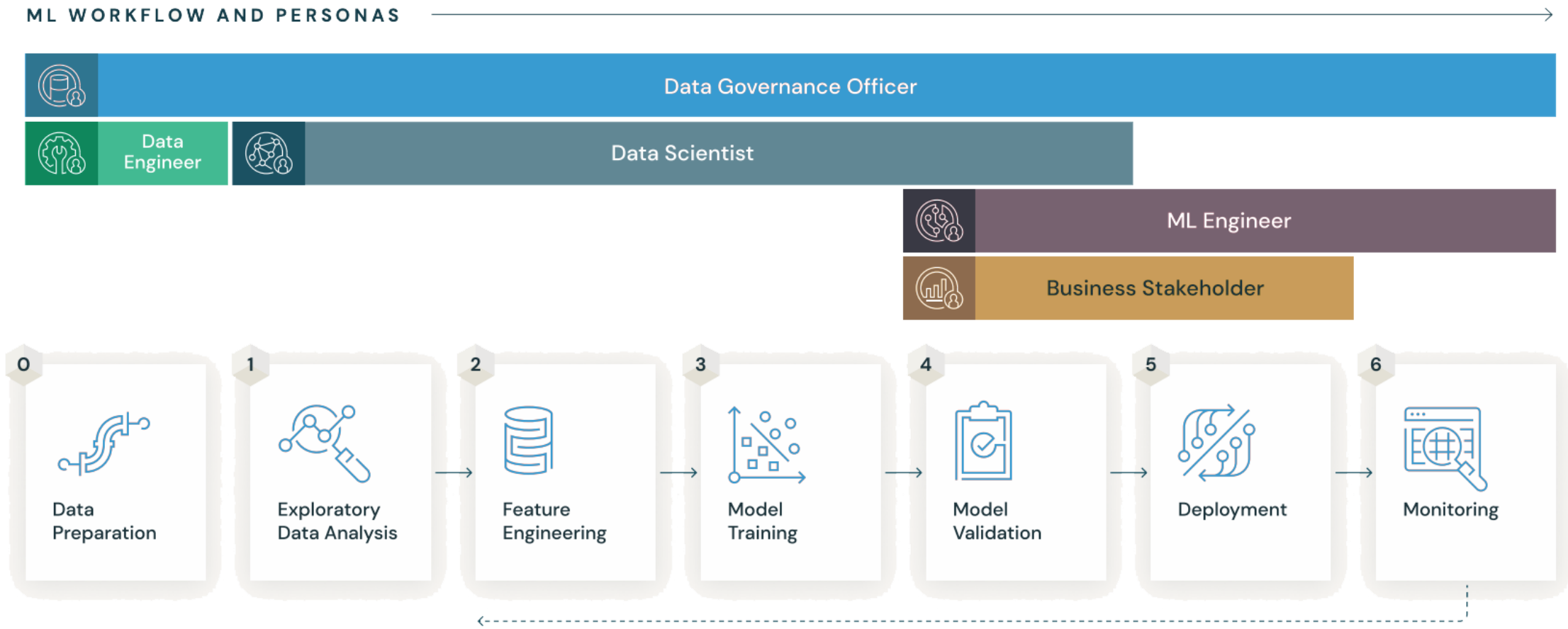
Model 1
Model 3
Model 2
Model 4

How to
- Tracking
- Manage
- Compare

Model 1
Model 3
Model 5
Model 2
Model 4
Model …

Introduction

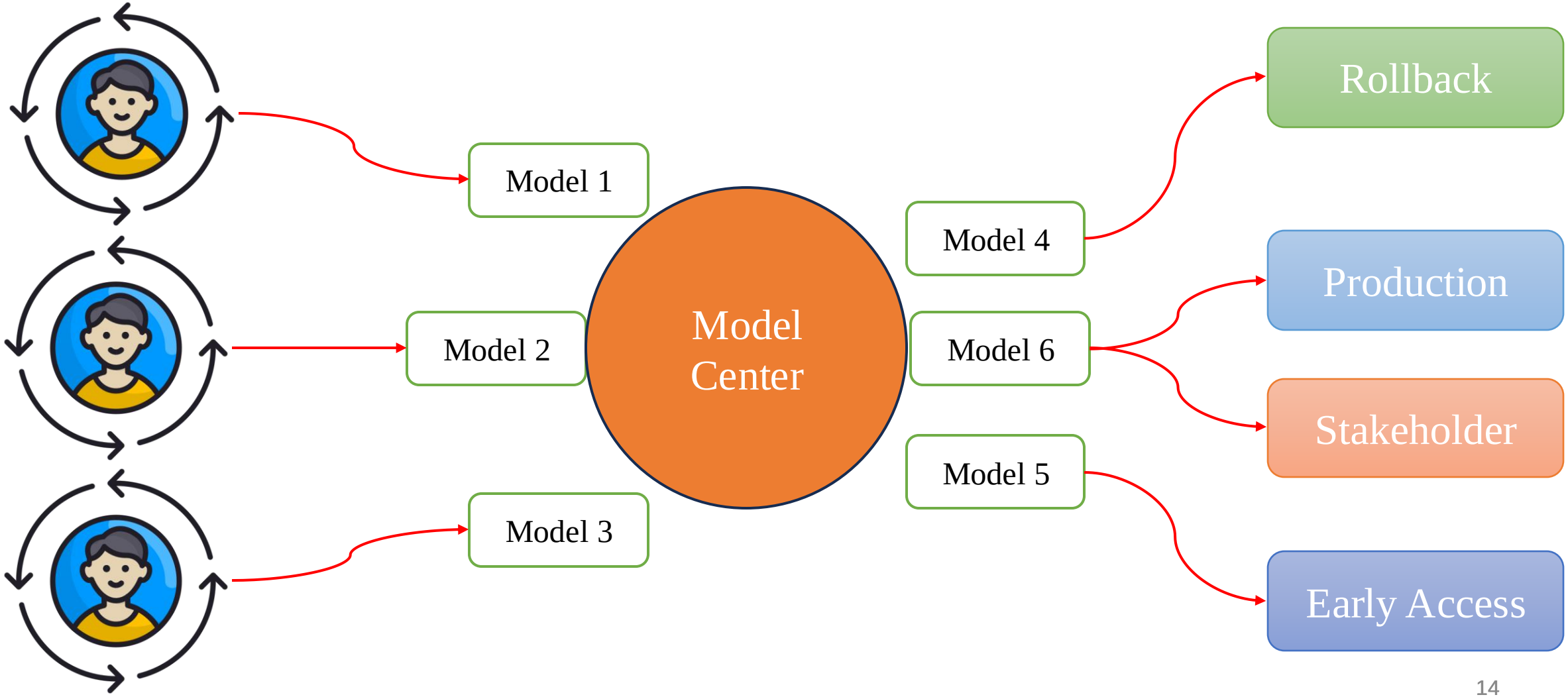
❖ Business-level



Ref: https://mlflow.org/docs/2.7.0/what-is-mlflow.html

Introduction

❖ Getting Started

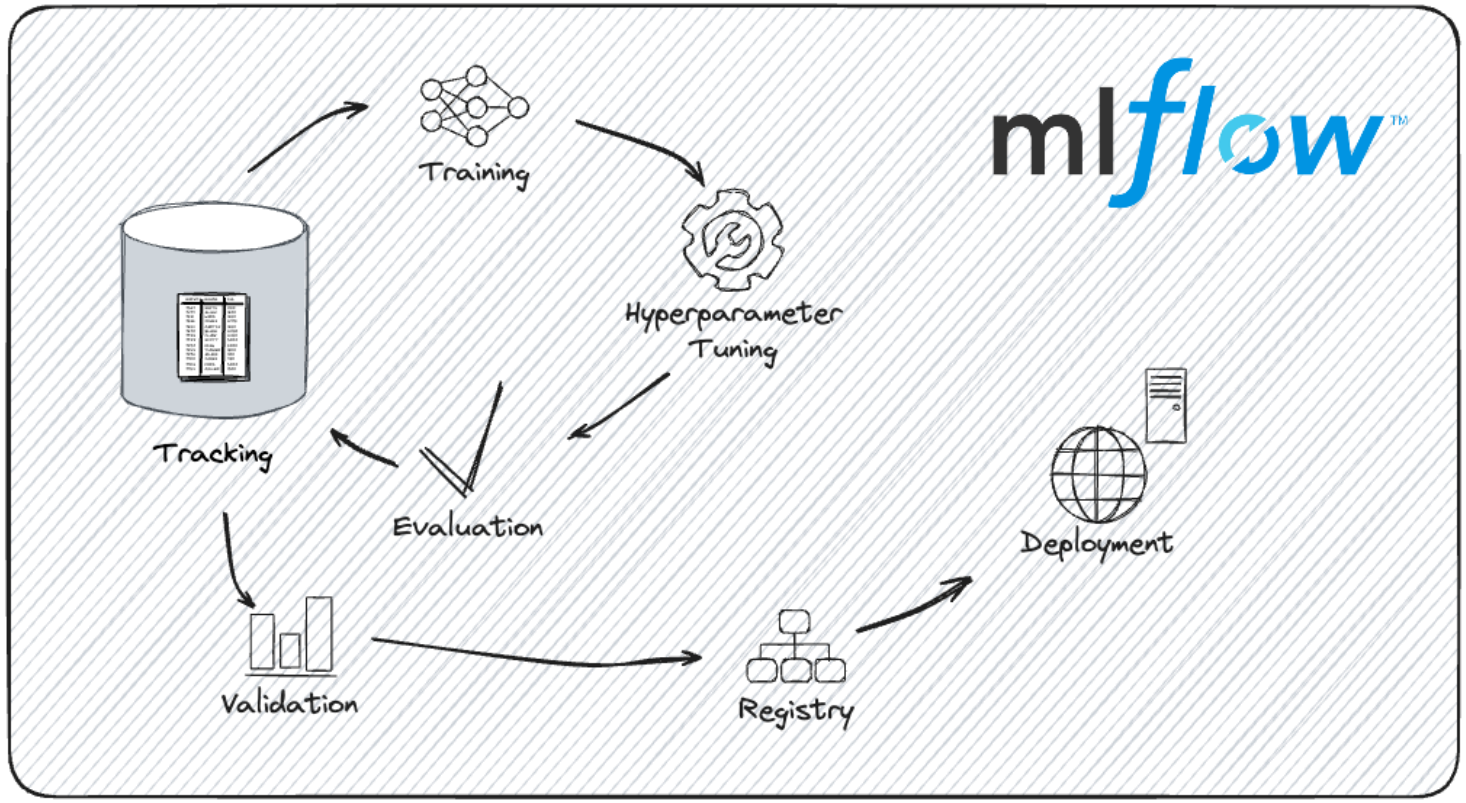


MLflow

MLflow

❖ Getting Started

- MLflow is an open-source platform designed to manage the complete machine learning lifecycle.
- MLflow focuses on the full lifecycle for machine learning projects, ensuring that each phase is manageable, traceable, and reproducible.

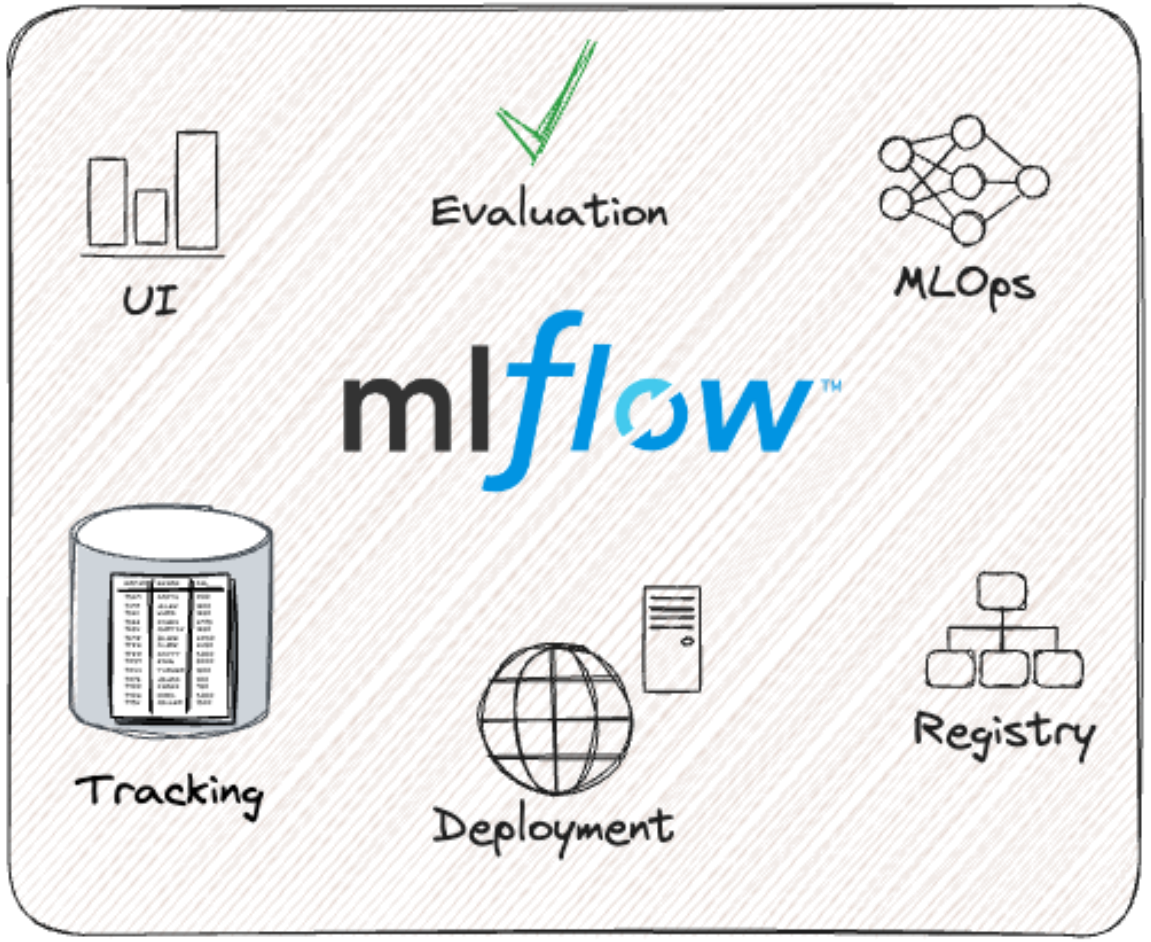


Ref: https://mlflow.org

MLflow

❖ Core Components

- MLFlow Tracking
- MLFlow Project
- MLFlow Model
- MLFlow Registry

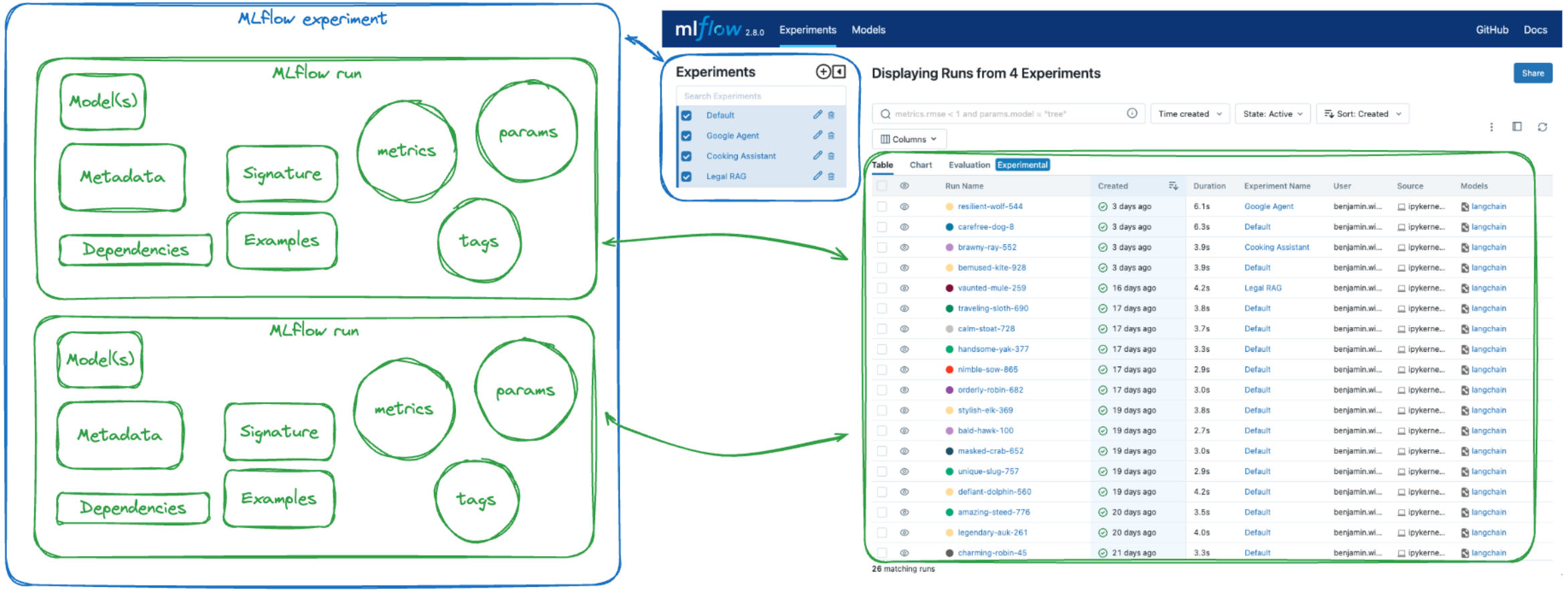


MLflow Tracking

MLflow Tracking

❖ MLflow Tracking

- An API and UI for logging params, metrics and artifacts of ML model.



MLflow Tracking

❖ Log params

- We usually log hyperparameters and training metrics using MLflow's tracking API.

```python
# data config
VERSION = "v0.1"
LOOKBACK = 14

# model config
input_size = 5
hidden_size = 128
num_layers = 2

# training config
SEED = 3
BATCH_SIZE = 16
LR = 1e-3
EPOCHS = 10
device = "cpu"
```

```python
import mlflow

with mlflow.start_run(run_name="rnn_model"):
    mlflow.log_params({
        "data_version": VERSION,
        "lookback": LOOKBACK,
        "input_size": input_size,
        "hidden_size": hidden_size,
        "num_layers": num_layers,
        "seed": SEED,
        "batch_size": BATCH_SIZE,
        "lr": LR,
        "num_epochs": EPOCHS,
        "device": device
    })
```

MLflow Tracking

❖ Log metrics

- **log_metric** is used to track the training loss at each epoch.
- Tracking further metrics such as accuracy or RMSE if needed

```python
# Training loop
for epoch in range(epochs):
    model.train()
    total_loss = 0
    for X_batch, y_batch in train_loader:
        optimizer.zero_grad()
        outputs = model(X_batch)
        loss = criterion(outputs, y_batch)
        loss.backward()
        optimizer.step()
        total_loss += loss.item()

    # Log the training loss for each epoch
    avg_loss = total_loss / len(train_loader)
    mlflow.log_metric("training_loss", avg_loss, step=epoch)
    print(f"Epoch [{epoch+1}/{epochs}], Loss: {avg_loss:.4f}")

```

MLflow Tracking

❖ Log artifacts

- **log_artifacts** is used to log a directory of files as artifacts.
- Artifacts are output files produced during an experiment, such as model weights, visualizations,…

```
model_path = "rnn_model.pth"
torch.save(model.state_dict(), model_path)
mlflow.log_artifact(model_path)
```

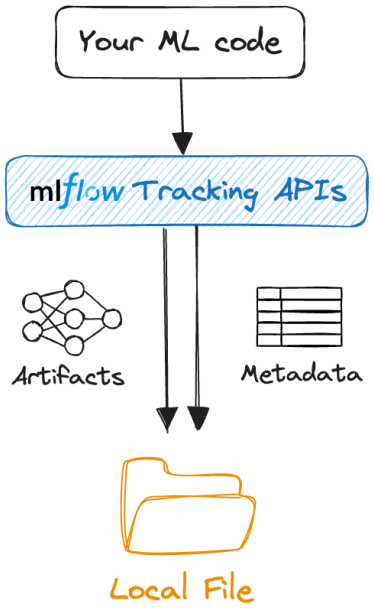
```
mlflow.log_artifact(path_save / "features_scaler.pkl", artifact_path="scalers")
mlflow.log_artifact(path_save / "target_scaler.pkl", artifact_path="scalers")
mlflow.log_artifact("./config/btc_config.yaml", artifact_path="config")

```

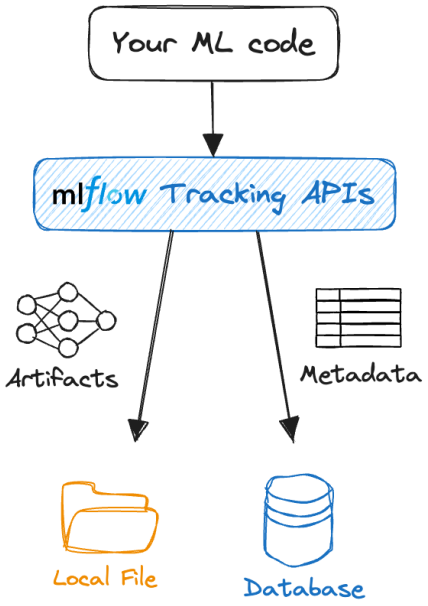
MLflow Tracking

❖ MLflow Tracking

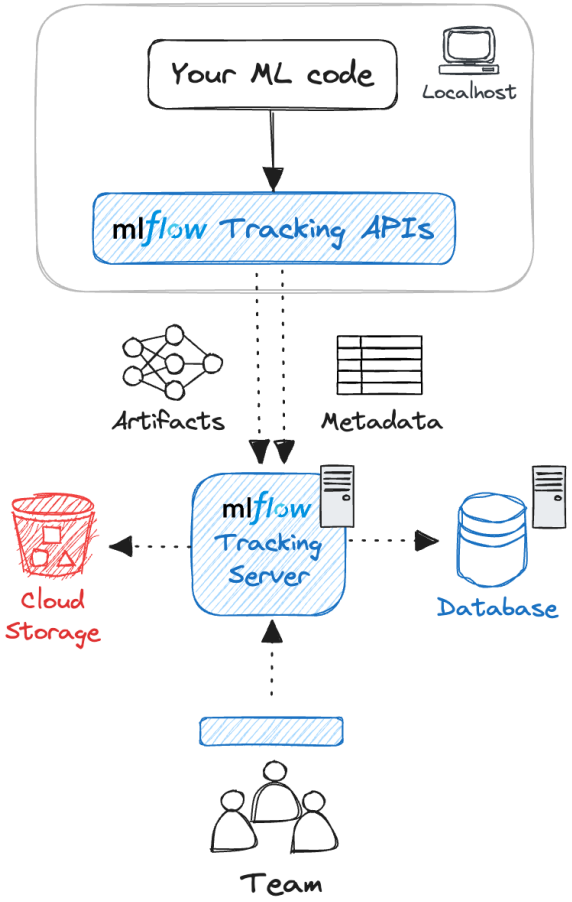
1. Localhost (default)



2. Localhost w/ various data stores



3. Remote Tracking w/ Tracking Server

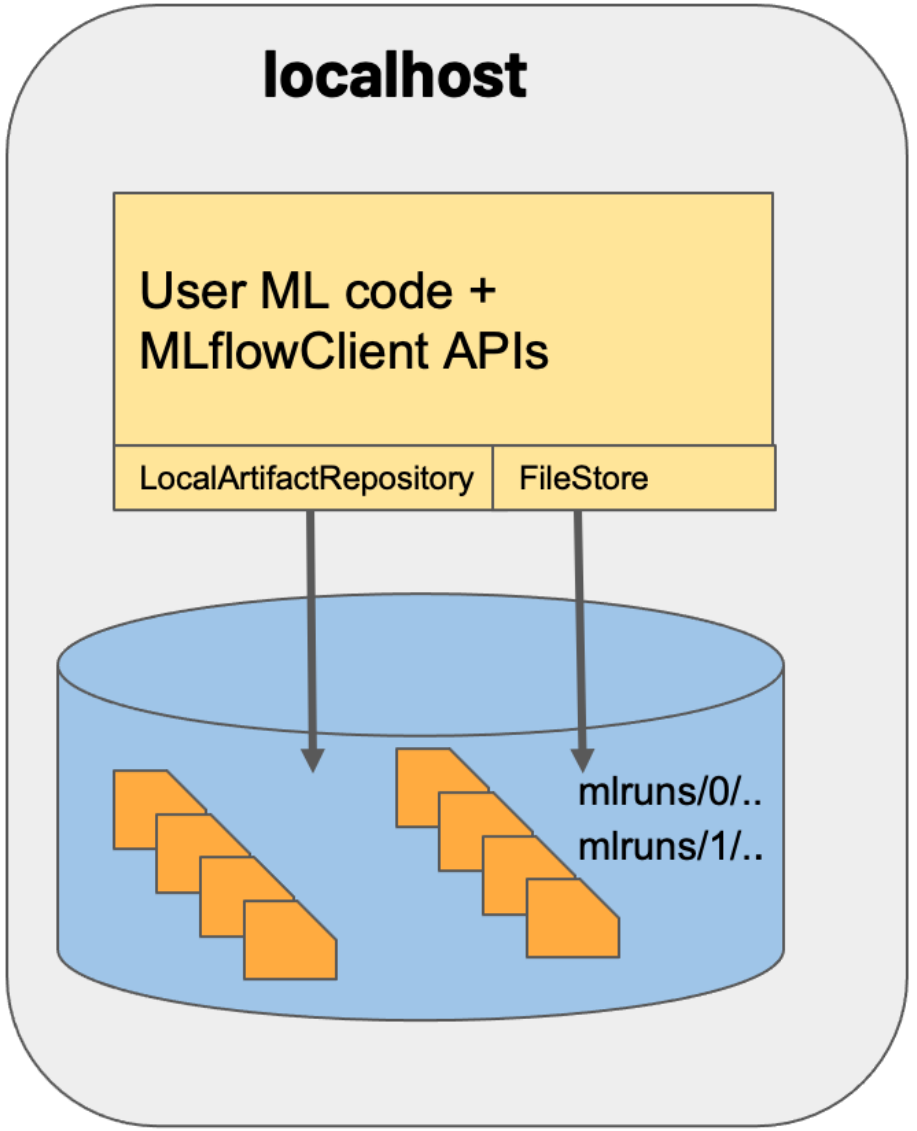


MLflow Tracking

❖ MLflow on localhost

- Run the MLflow server on your local machine.
- The MLflow Tracking Server runs locally, and the default backend store is a file-based system.

Use Case: Ideal for single-user scenarios or quick local experimentation without any additional database setup



Ref: https://mlflow.org/docs/2.2.2/tracking.html#how-runs-and-artifacts-are-recorded

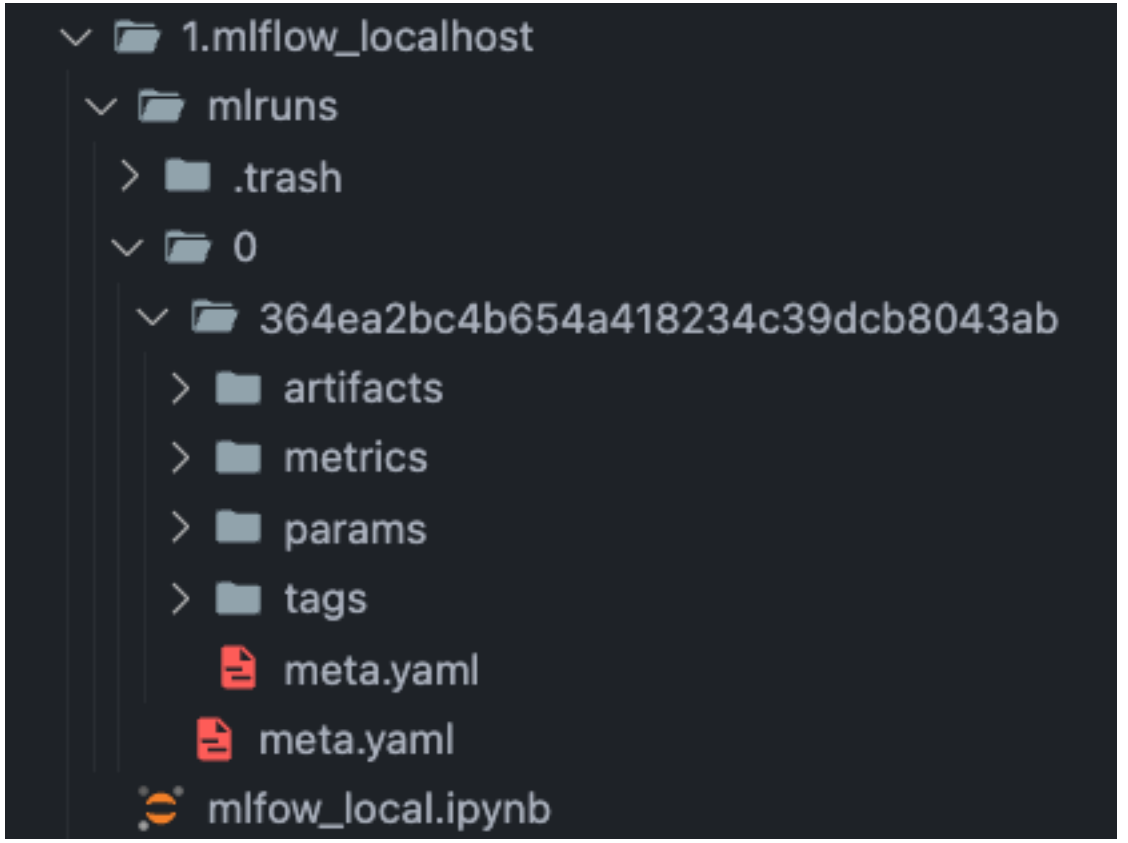
MLflow Tracking

❖ MLflow on localhost

```python
import mlflow

with mlflow.start_run(run_name="rnn_model"):
    mlflow.log_params({
        "data_version": VERSION,
        "lookback": LOOKBACK,
        "input_size": input_size,
        "hidden_size": hidden_size,
        "num_layers": num_layers,
        "seed": SEED,
        "batch_size": BATCH_SIZE,
        "lr": LR,
        "num_epochs": EPOCHS,
        "device": device
    })
```



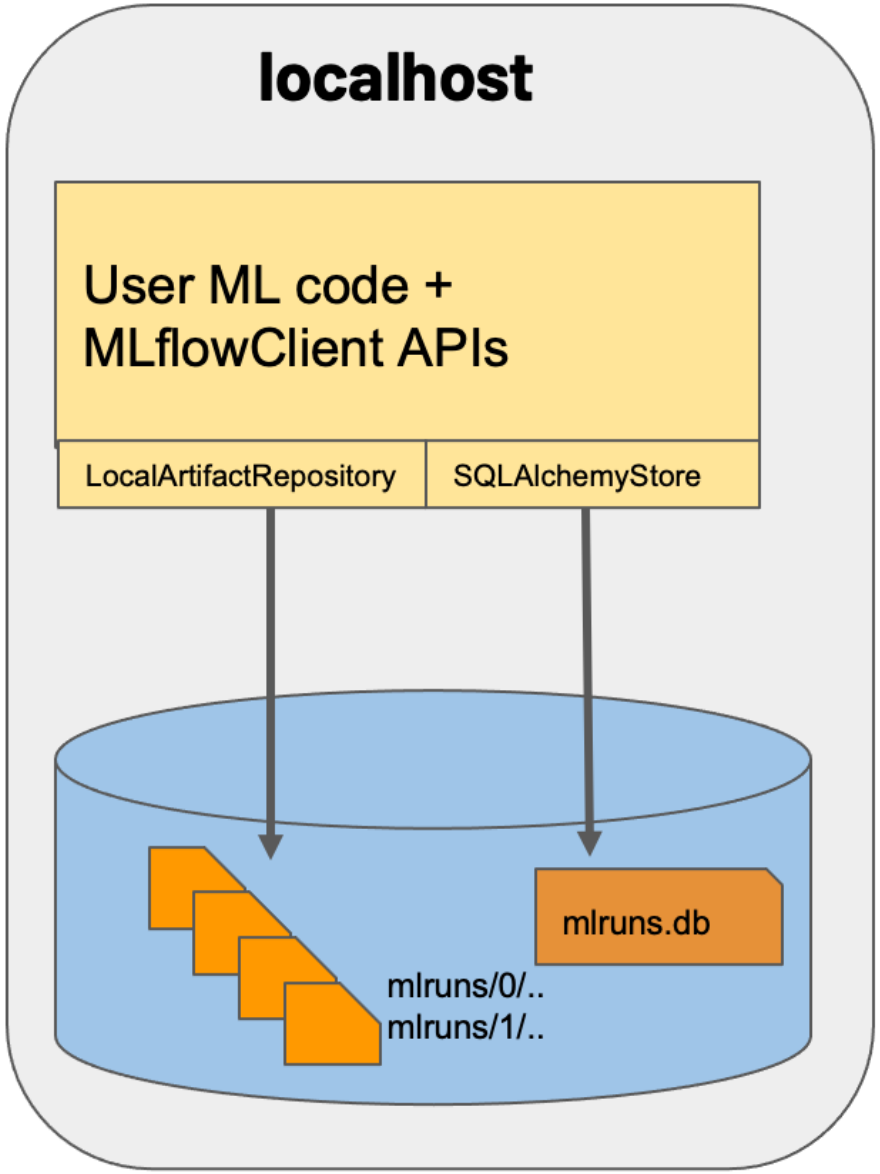


MLflow Tracking

❖ MLflow on localhost with sqlite

- Using SQLite as the backend metadata store for the MLflow Tracking Server.
- Still runs locally but uses an SQLite database (.db file) to store metadata.

Use Case: A more robust option for small-scale teams or projects.



Ref: https://mlflow.org/docs/2.2.2/tracking.html#how-runs-and-artifacts-are-recorded

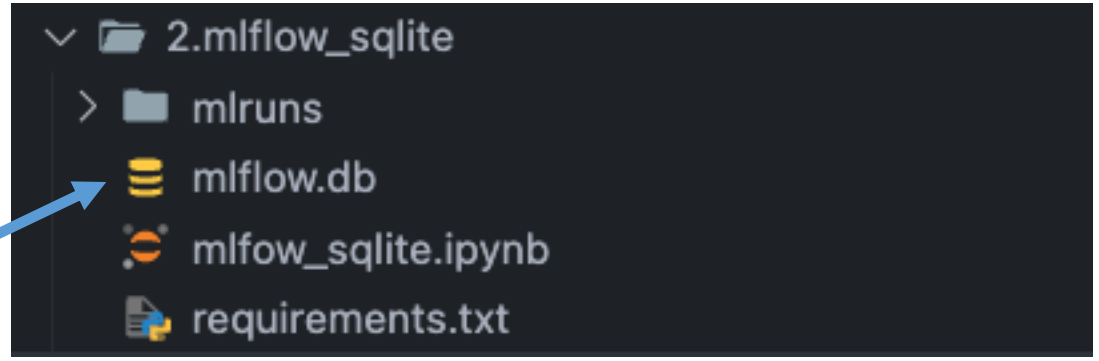
MLflow Tracking

❖ MLflow on localhost with sqlite

```python
import mlflow

mlflow.set_tracking_uri("sqlite:///mlflow.db")

with mlflow.start_run(run_name="rnn_model"):
    mlflow.log_params({
        "data_version": VERSION,
        "lookback": LOOKBACK,
        "input_size": input_size,
        "hidden_size": hidden_size,
        "num_layers": num_layers,
        "seed": SEED,
        "batch_size": BATCH_SIZE,
        "lr": LR,
        "num_epochs": EPOCHS,
        "device": device
    })
```



MLflow Tracking

❖ MLflow on localhost with sqlite

- Query to sqlite database to see logging metadata

```python
import sqlite3
import pandas as pd

# Connect to the SQLite database
conn = sqlite3.connect("mlflow.db")

# Query experiments table
experiments_query = "SELECT * FROM experiments"
experiments = pd.read_sql_query(experiments_query, conn)

# Query runs table
runs_query = "SELECT * FROM runs"
runs = pd.read_sql_query(runs_query, conn)

# Close the connection
conn.close()
```

```
experiments
[5] ✓ 0.0s
```

	experiment_id	name	artifact_location	lifecycle_stage	creation_time	last_update_time
0	0	Default	/Users/thuanduong/AIVietnam/TA/MLOPS/06_MLOps_...	active	1733877943071	1733877943071

```
runs
[6] ✓ 0.0s
```

	run_uuid	name	source_type	source_name	entry_point_name	user_id	status	start_time	end_time
0	698cd33c662543748e98d0c030fd1941	rnn_model	UNKNOWN			thuanduong	FINISHED	1733877943128	1733877943137

MLflow Tracking

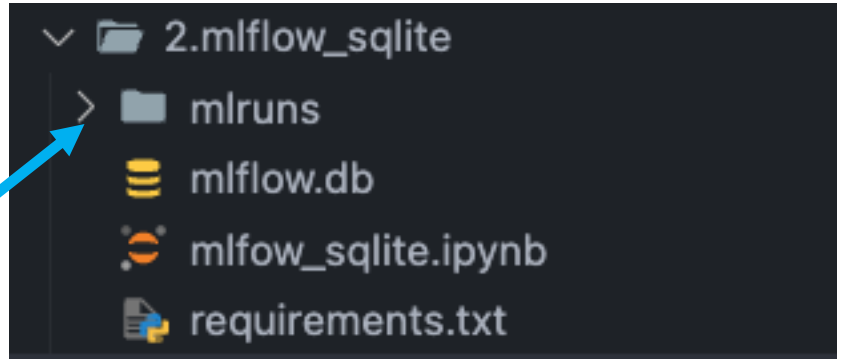
❖ MLflow on localhost with sqlite

```python
epoch_loss = running_loss / len(train_loader)
mlflow.log_metric("training_mse_loss", f"{epoch_loss:6f}", step=epoch)

val_loss = evaluate(model, val_loader)
mlflow.log_metric("val_mse_loss", f"{val_loss:.6f}", step=epoch)

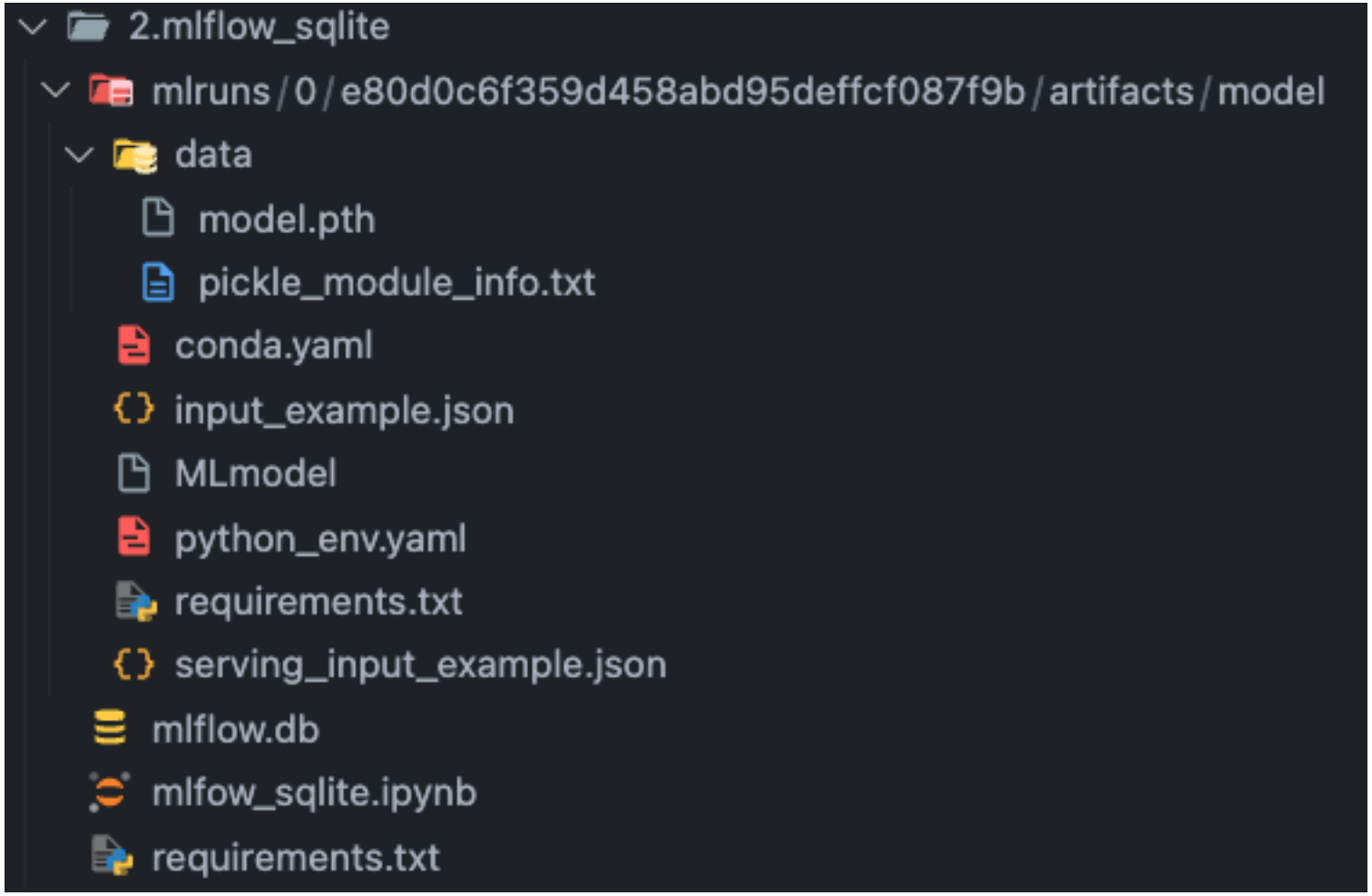
if val_loss < best_loss:
    best_loss = val_loss
    best_model_state_dict = model.state_dict()

model.load_state_dict(best_model_state_dict)
input_example = torch.rand(1, LOOKBACK, input_size).numpy()
mlflow.pytorch.log_model(model,
                         artifact_path="model",
                         pip_requirements="./requirements.txt",
                         input_example=input_example)
```



MLflow Tracking

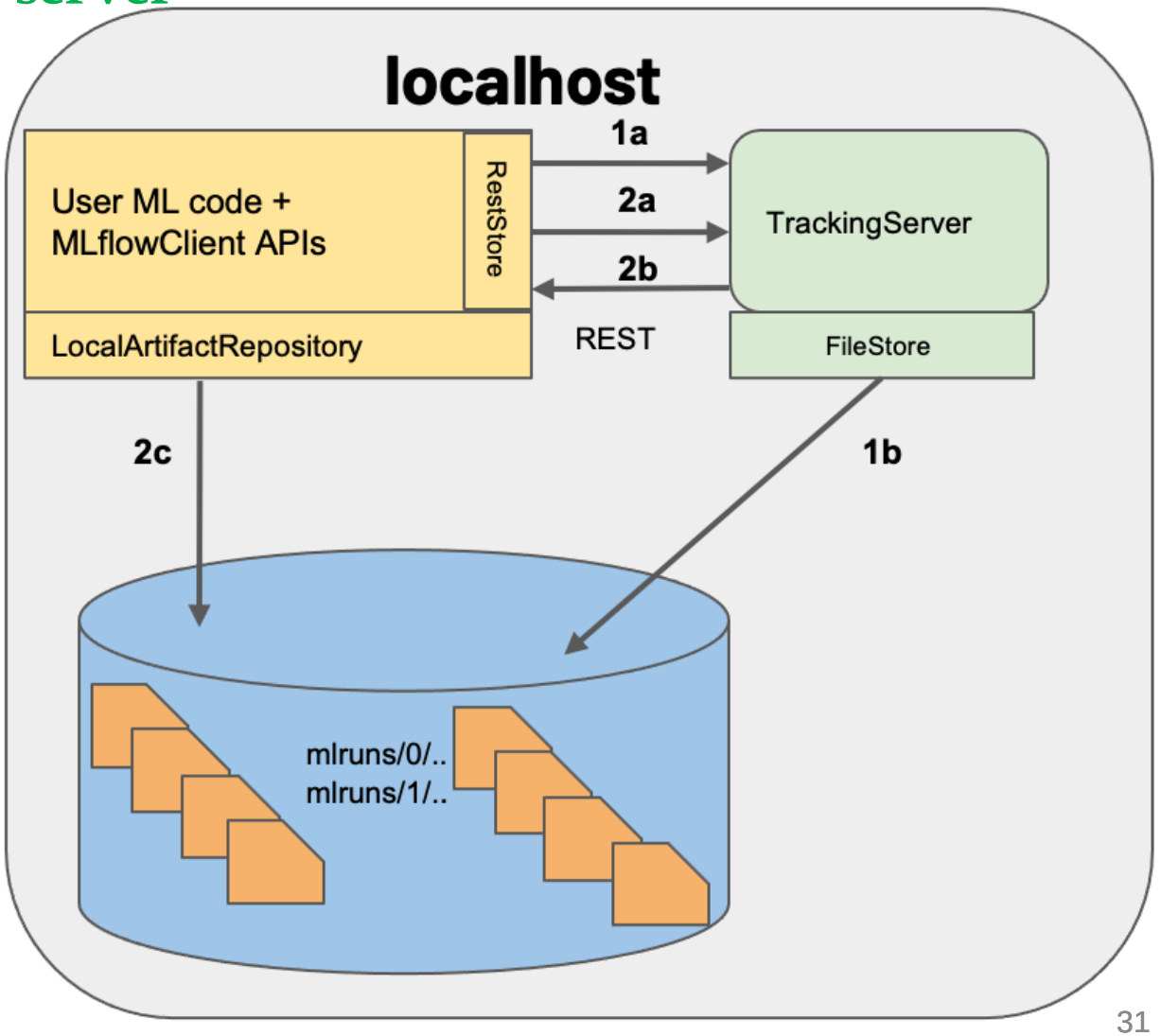
❖ MLflow on localhost with sqlite



MLflow Tracking

❖ MLflow on localhost with tracking server

- The server is hosted on your local machine.
- More options for backend metadata store (e.g., file system, SQLite, PostgreSQL) and artifact storage (e.g., local file system, S3, etc.).



Use Case: Suitable for teams and projects requiring centralized tracking, flexibility in storage configuration, and better scalability.

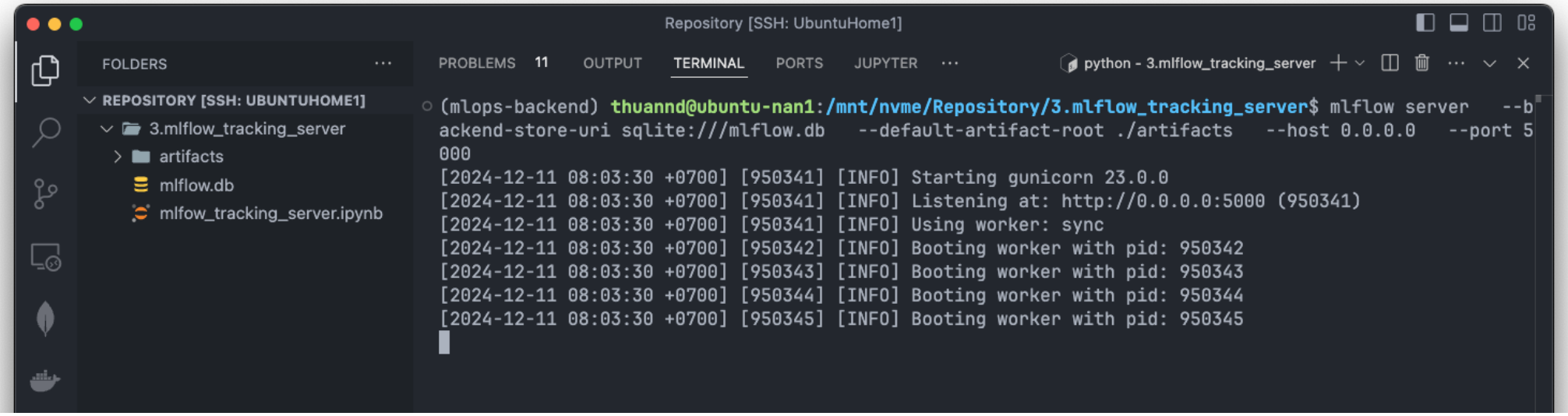
Ref: https://mlflow.org/docs/2.2.2/tracking.html#how-runs-and-artifacts-are-recorded

MLflow Tracking

❖ MLflow on localhost with tracking server

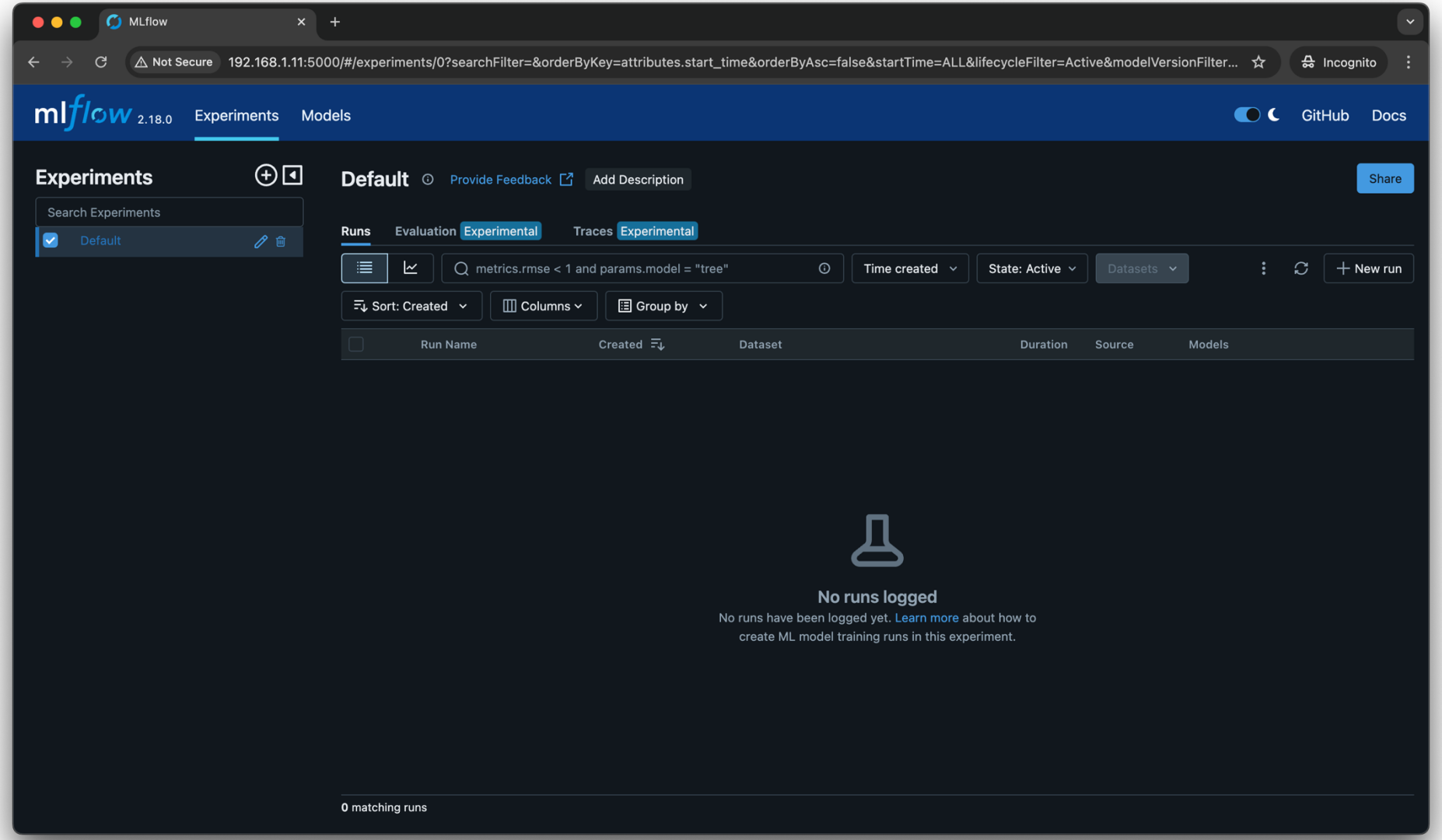
```
mlflow server \
    --backend-store-uri sqlite:///mlflow.db \
    --default-artifact-root ./artifacts \
    --host 0.0.0.0 \
    --port 5000
```

Start MLflow tracking server



MLflow Tracking

❖ MLflow on localhost with tracking server



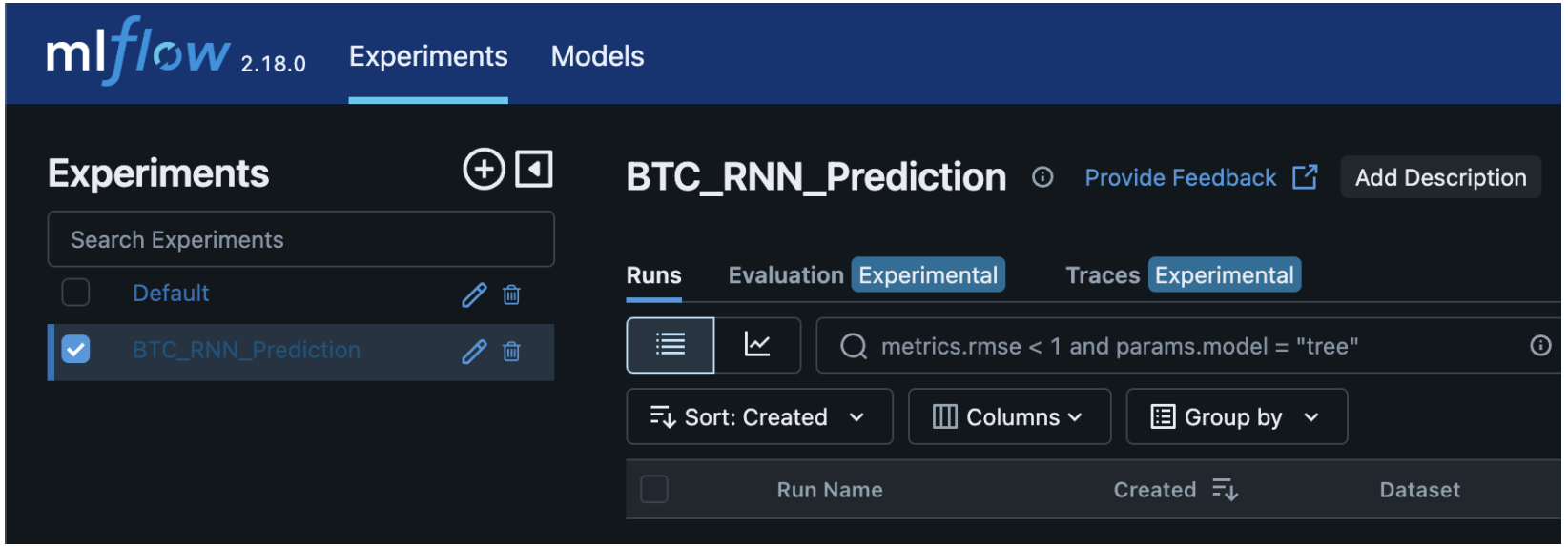
MLflow Tracking

❖ MLflow on localhost with tracking server

```python
import mlflow

mlflow.set_tracking_uri("http://192.168.1.11:5000")
mlflow.set_experiment("BTC_RNN_Prediction")
```

Create an experiment



MLflow Tracking

❖ MLflow on localhost with tracking server

Logging params, metrics and artiofacts to MLflow tracking server

```python
with mlflow.start_run(run_name="rnn_base"):
    mlflow.log_params({
        "data_version": VERSION,
        "lookback": LOOKBACK,
        "input_size": input_size,
        "hidden_size": hidden_size,
        "num_layers": num_layers,
        "seed": SEED,
        "batch_size": BATCH_SIZE,
        "lr": LR,
        "num_epochs": EPOCHS,
        "device": device
    })
```

```python
epoch_loss = running_loss / len(train_loader)
mlflow.log_metric("training_mse_loss", f"{epoch_loss:6f}", step=epoch)

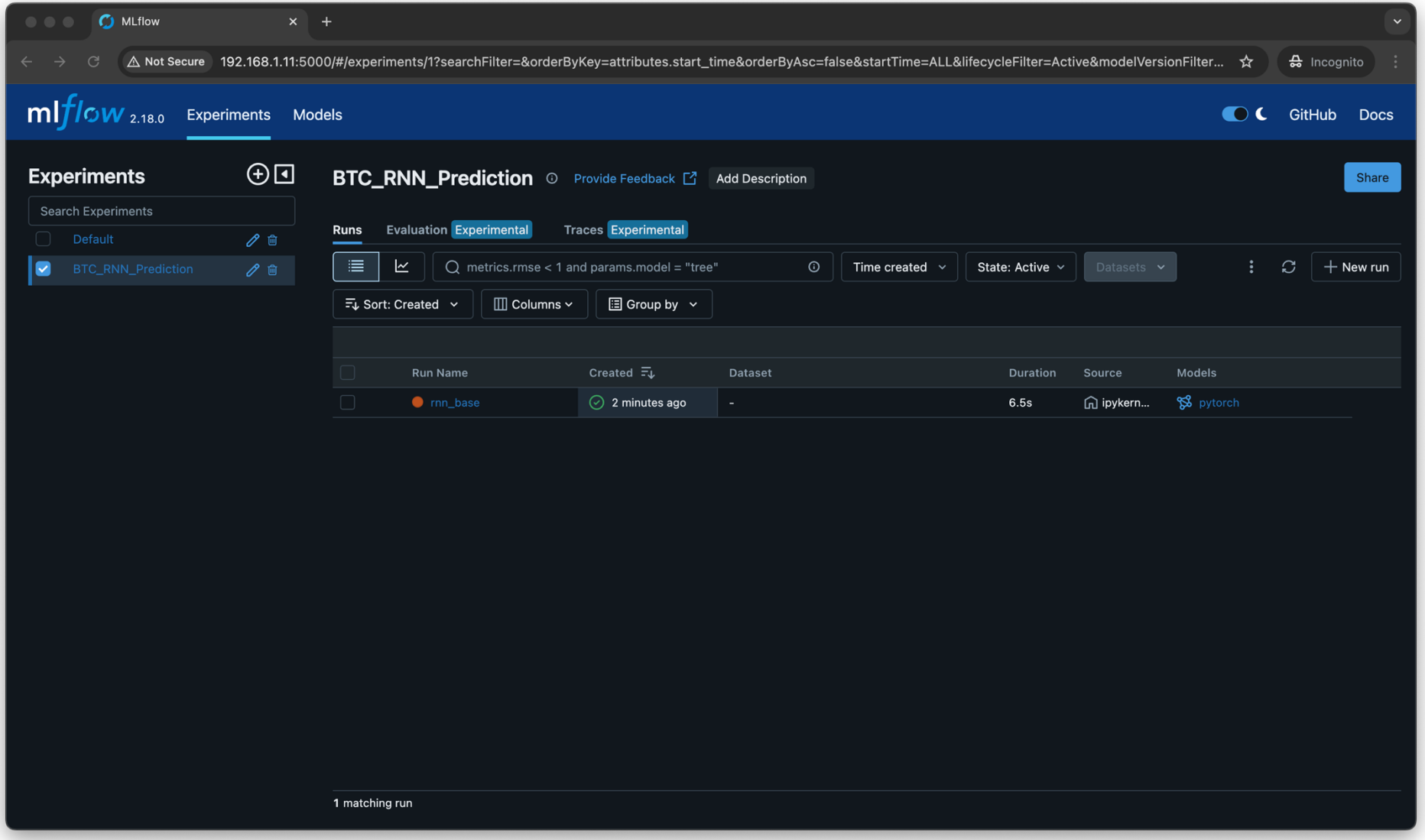
val_loss = evaluate(model, val_loader)
mlflow.log_metric("val_mse_loss", f"{val_loss:.6f}", step=epoch)

if val_loss < best_loss:
    best_loss = val_loss
    best_model_state_dict = model.state_dict()

model.load_state_dict(best_model_state_dict)
input_example = torch.rand(1, LOOKBACK, input_size).numpy()
mlflow.pytorch.log_model(model,
                         artifact_path="model",
                         pip_requirements="./requirements.txt",
                         input_example=input_example)
```

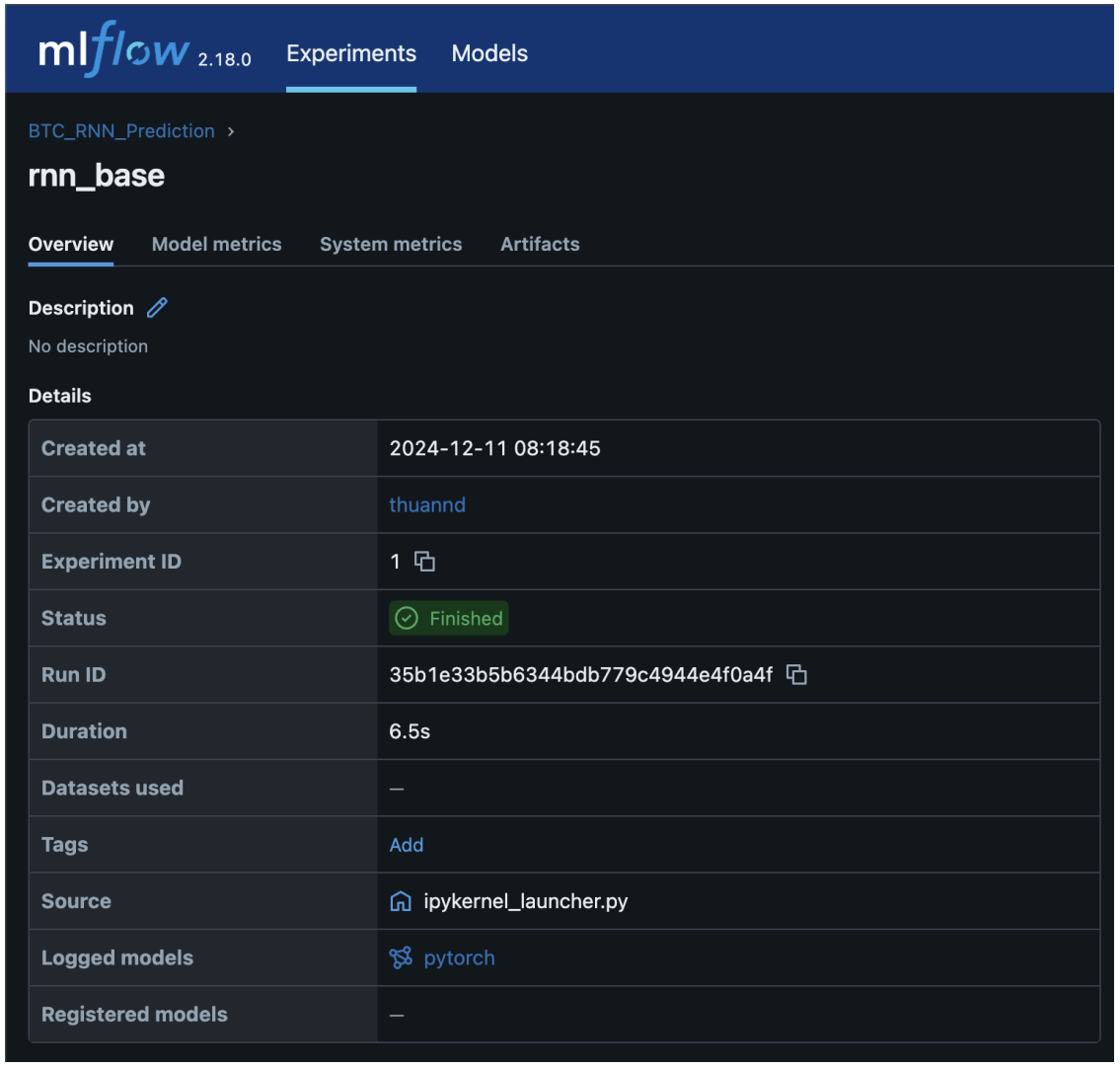
MLflow Tracking

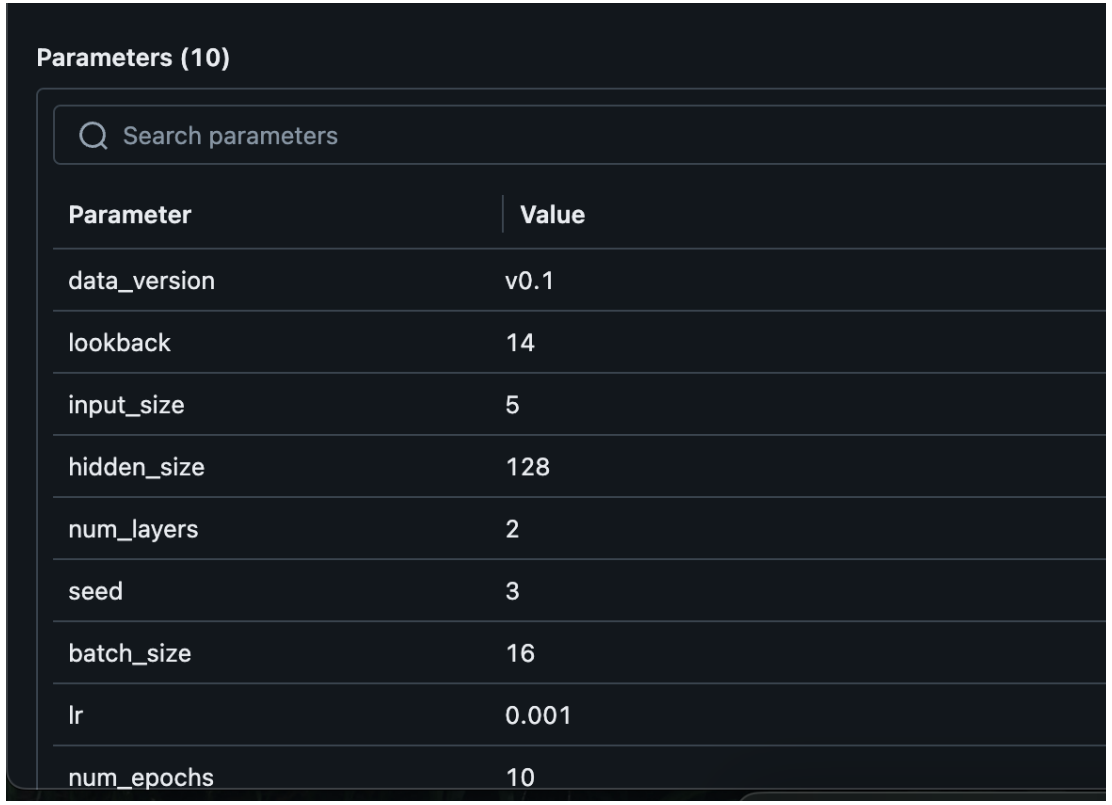
❖ MLflow on localhost with tracking server



MLflow Tracking

❖ MLflow on localhost with tracking server







MLflow Tracking

❖ Summary

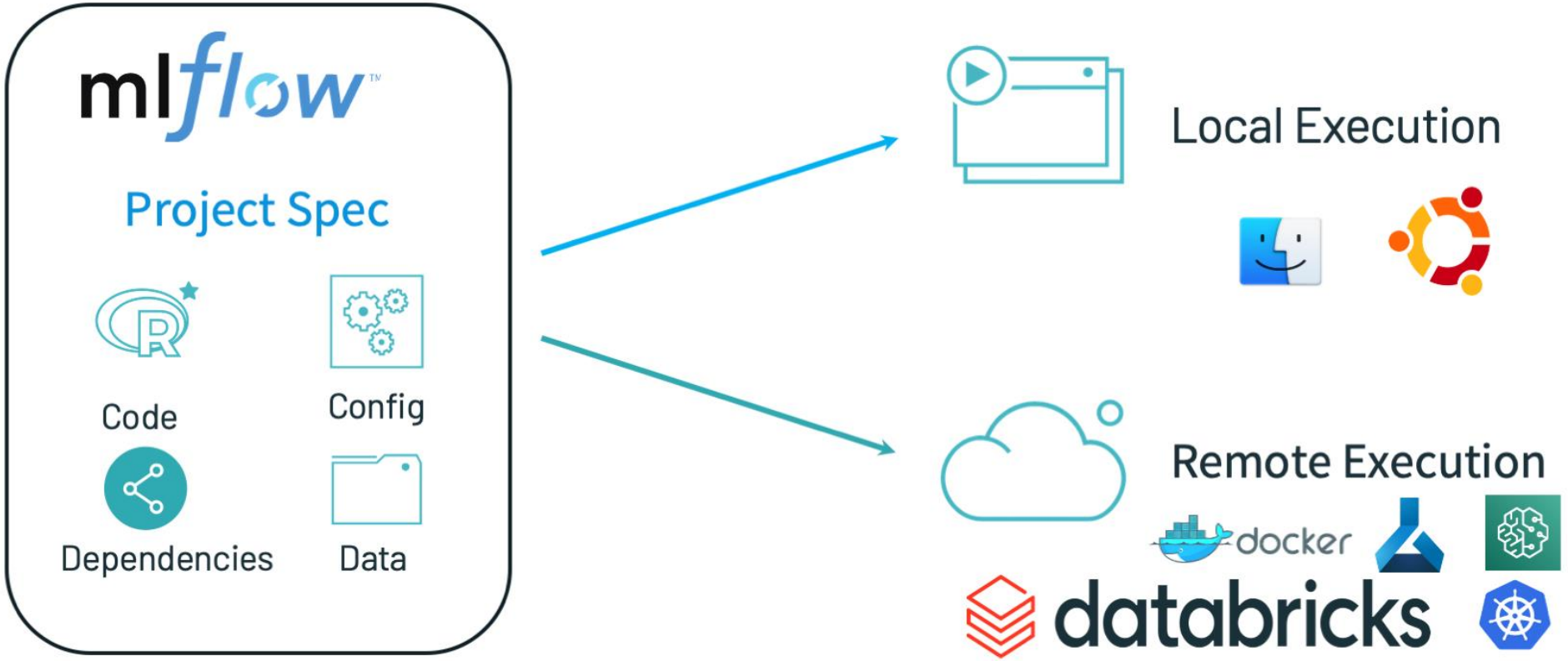
	Local disk	SQLite	Tracking Server
Complexity	Low	Moderate	High
Scalability	Limited	Moderate	High
Collaboration	Minimal	Basic	Excellent
Artifact Management	Local only	Local/Remote	Local/Remote
Concurrency	Single user	Limited	Multi-user support
Best For	Personal	Small Teams	Production, Large Teams

MLflow Project

MLflow Project

❖ Getting Started

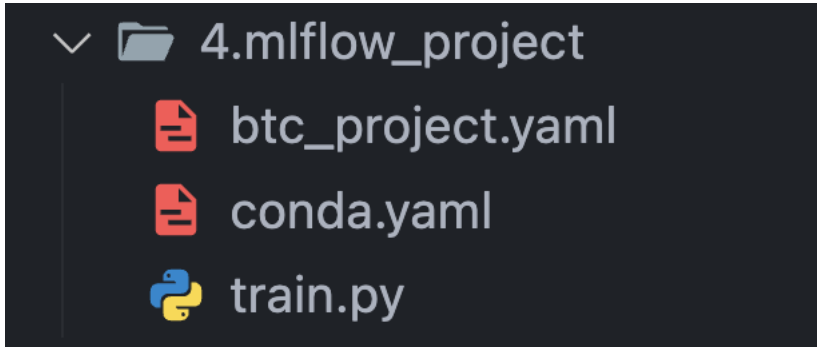
- Package data science code in a standard format that enables reusable and runs one many platform.



Ref: https://www.datacamp.com/tutorial/mlflow-streamline-machine-learning-workflow

MLflow Project

❖ Project structure




```yaml
name: btc_prediction

entry_points:
  main:
    parameters:
      batch_size: {type: int, default: 16}
      epochs: {type: int, default: 10}
    command: "python train.py --batch_size {batch_size} --epochs {epochs}"

conda_env: conda.yaml
```

```yaml
channels:
- conda-forge
dependencies:
- python=3.11.9
- pip<=24.2
- pip:
    - mlflow==2.18.0
    - torch==2.2.2
    - pandas==2.2.3
    - numpy==1.26.1
    - scikit-learn==1.5.2
name: mlflow-env
```

MLflow Project

❖ Project structure

```python
if __name__ == '__main__':
    parser = argparse.ArgumentParser(description="Train a model to predict BTC price")

    # data config
    parser.add_argument('--data_version', type=str, default='v0.1', help='Data version')
    parser.add_argument('--lookback', type=int, default=14, help='Lookback window')

    # model config
    parser.add_argument('--input_size', type=int, default=5, help='Input size of RNN')
    parser.add_argument('--hidden_size', type=int, default=128, help='Hidden size of RNN')
    parser.add_argument('--num_layers', type=int, default=2, help='Number of layers of RNN')

    # training config
    parser.add_argument('--seed', type=int, default=3, help='Random seed')
    parser.add_argument('--batch_size', type=int, default=16, help='Batch size')
    parser.add_argument('--lr', type=float, default=1e-3, help='Learning rate')
    parser.add_argument('--epochs', type=int, default=10, help='Number of epochs')
    parser.add_argument('--device', type=str, default='cpu', help='Device')

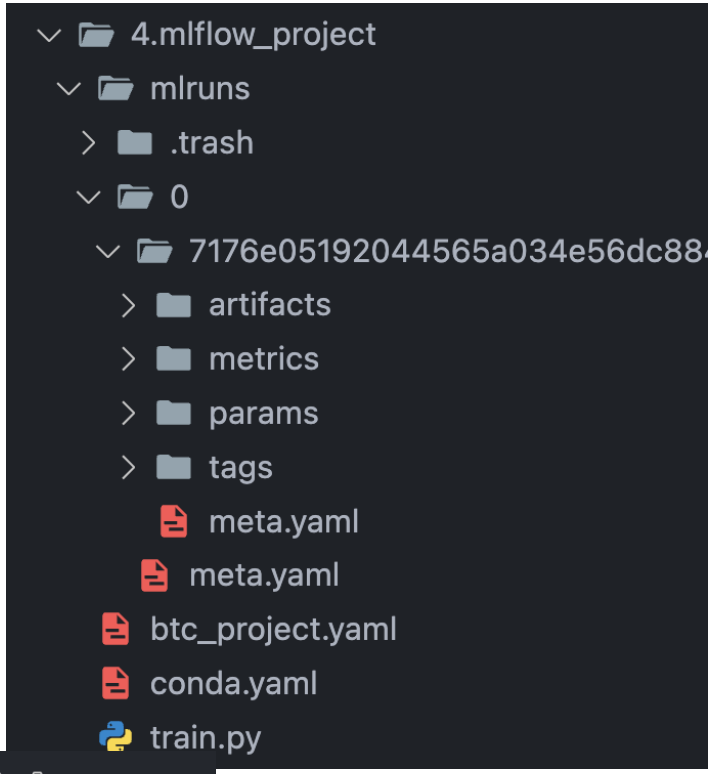
    args = parser.parse_args()

    train_model(args)
```

MLflow Project

❖ Run project

mlflow run . -e train.py



PROBLEMS OUTPUT TERMINAL PORTS DEBUG CONSOLE zsh - 4.mlflow_project

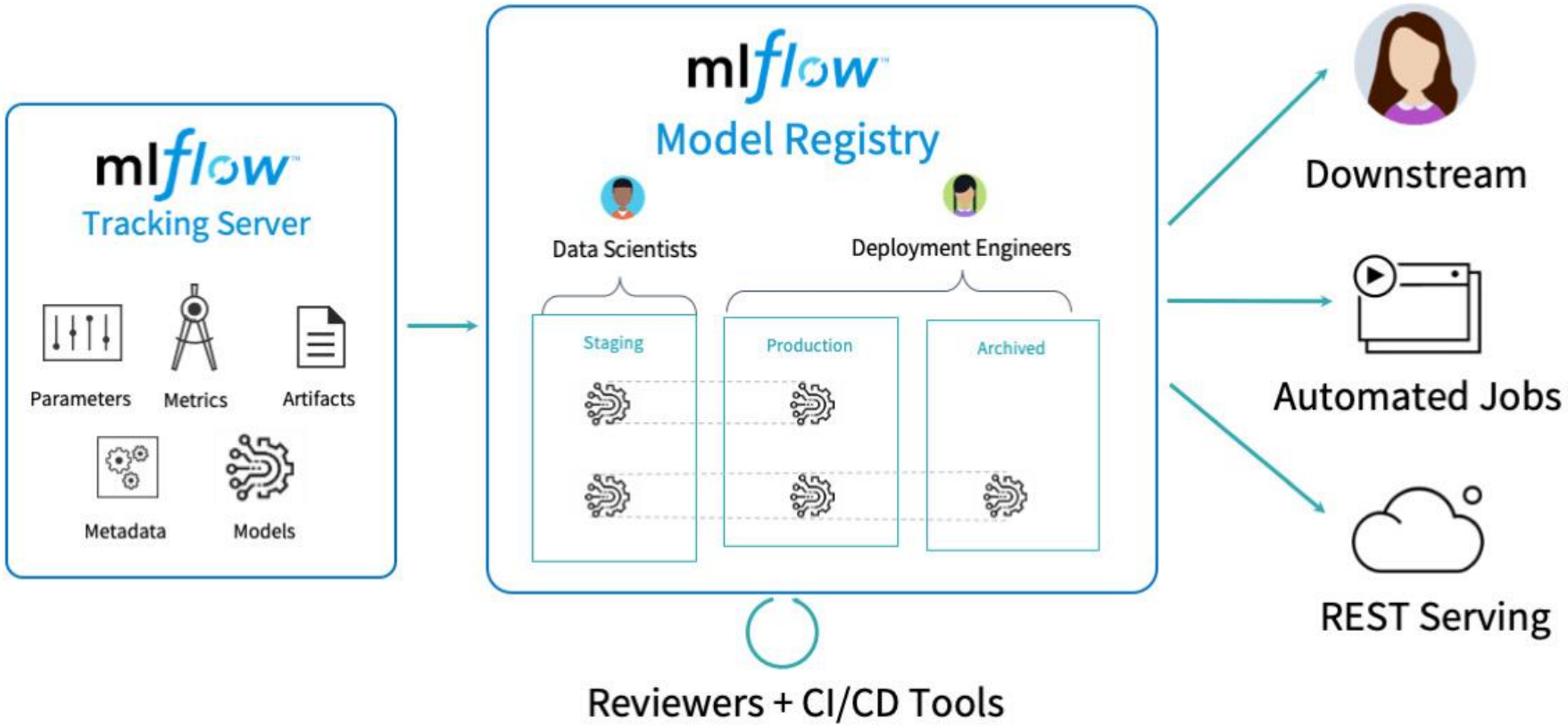
```
(mlops) thuanduong@MacNaN 4.mlflow_project % mlflow run . -e train.py
2024/12/11 12:25:13 INFO mlflow.utils.conda: Conda environment mlflow-15b0f63de20f73f9bf9bfb39bf195a41734d2af0
 already exists.
2024/12/11 12:25:13 INFO mlflow.projects.utils: === Created directory /var/folders/h8/cym9q84x4fd_2yxd4m1lt33r
0000gn/T/tmp0k6kx3ru for downloading remote URIs passed to arguments of type 'path' ===
2024/12/11 12:25:13 INFO mlflow.projects.backend.local: === Running command 'source /Users/thuanduong/minicond
a3/bin/../etc/profile.d/conda.sh && conda activate mlflow-15b0f63de20f73f9bf9bfb39bf195a41734d2af0 1>&2 && pyt
hon train.py' in run with ID '7176e05192044565a034e56dc884426e' ===
2024/12/11 12:25:31 INFO mlflow.projects: === Run (ID '7176e05192044565a034e56dc884426e') succeeded ===
(mlops) thuanduong@MacNaN 4.mlflow_project %
```

MLflow Registry

MLflow Registruy

❖ MLflow Registry

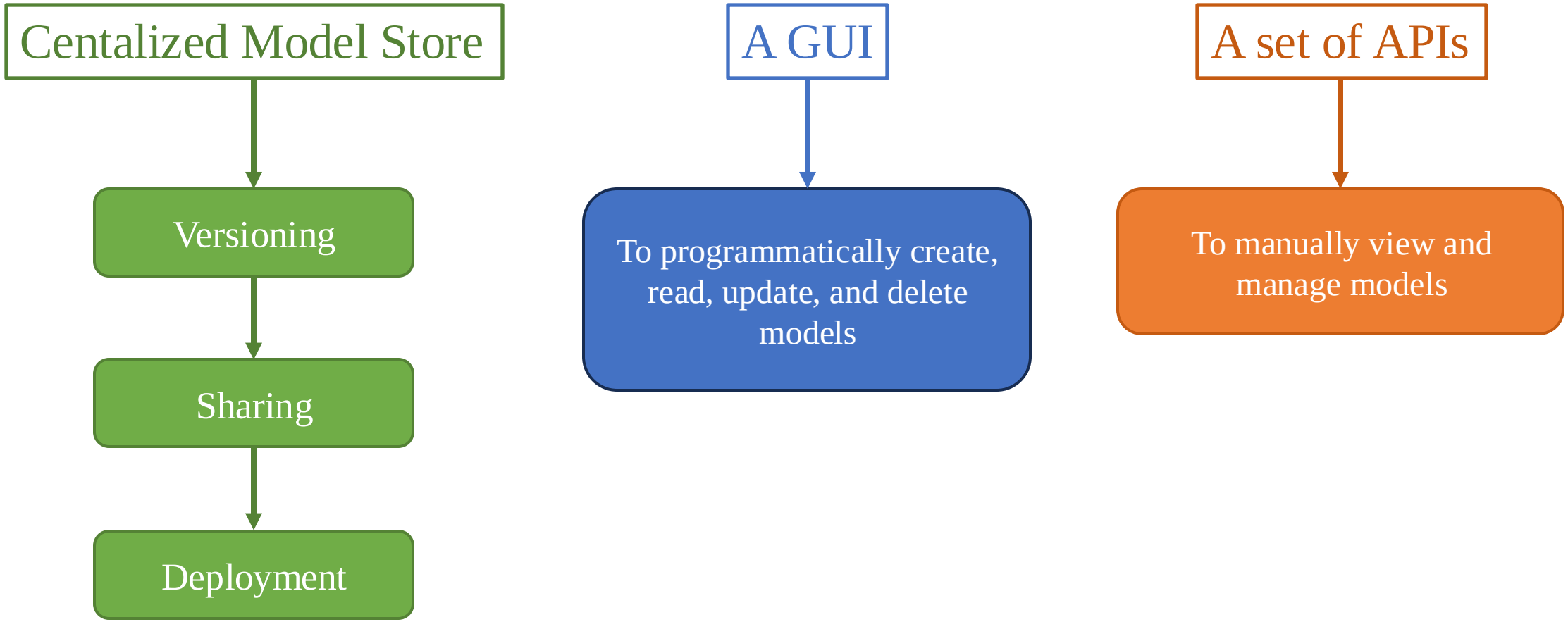
- MLflow Registry: offers a centralized model store, set of APIs, and UI, to collaboratively manage the full lifecycle of an MLflow Model.



Ref: https://mlflow.org/docs/latest/model-registry.html

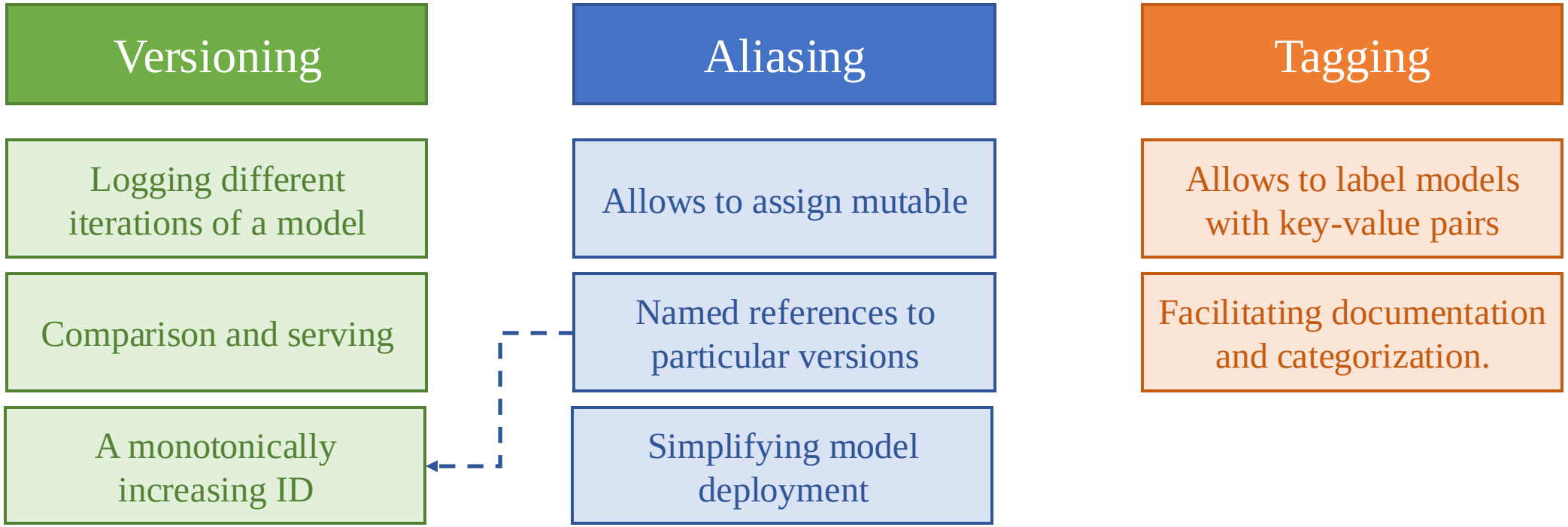
MLflow Registruy

❖ Components



MLflow Registruy

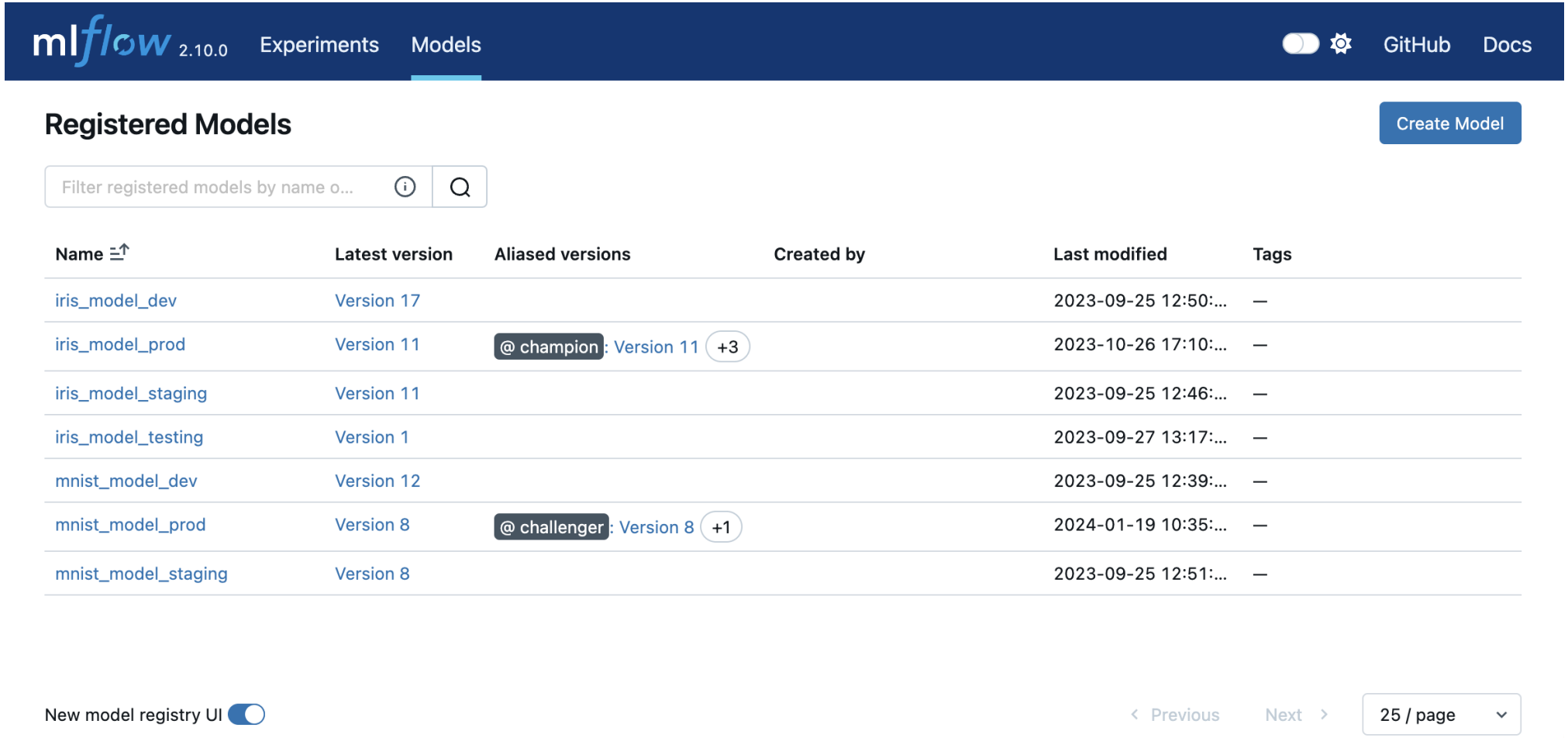
❖ Versioning, Aliasing, Tagging



MLflow Registruy

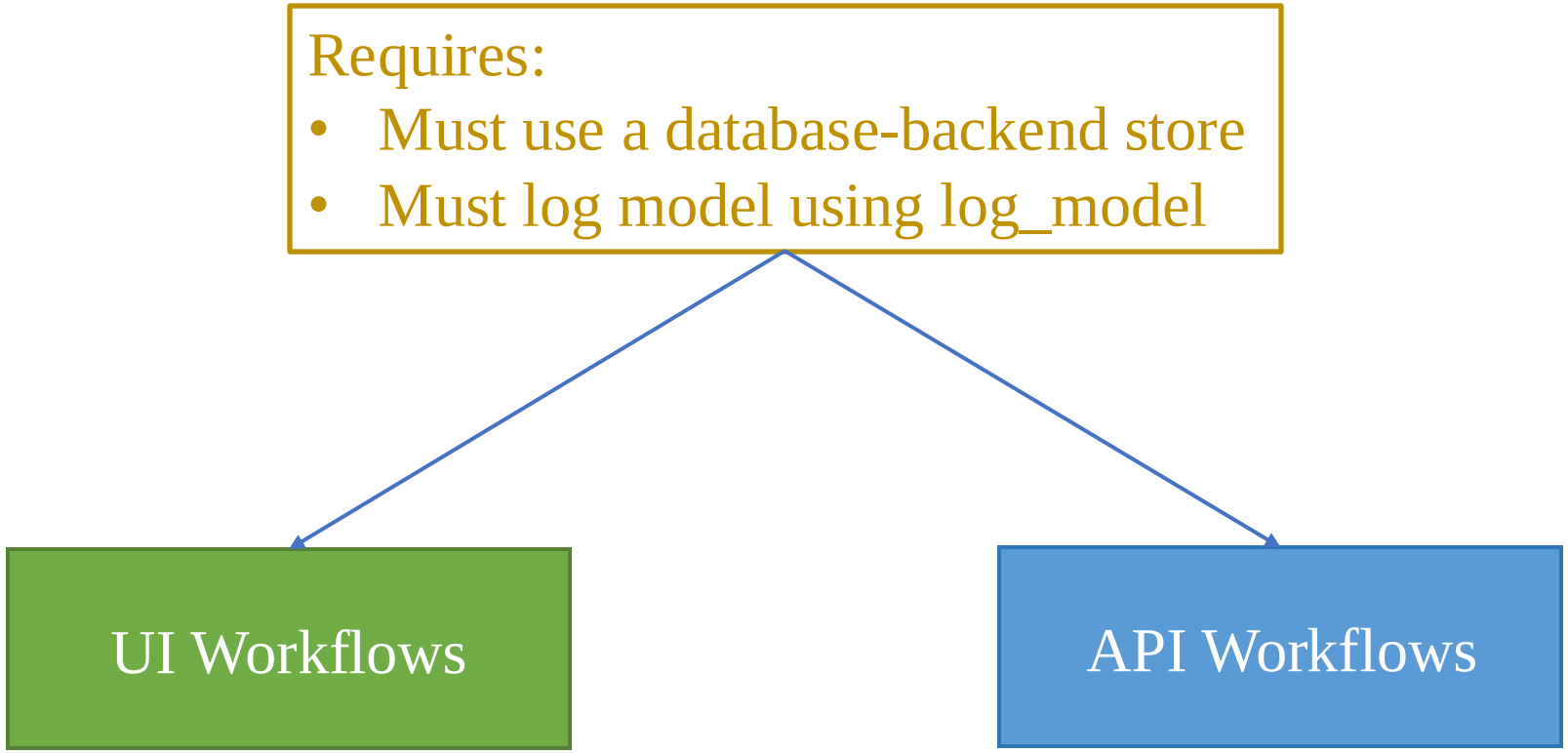
❖ GUI

- Tracks different versions and aliased of model.



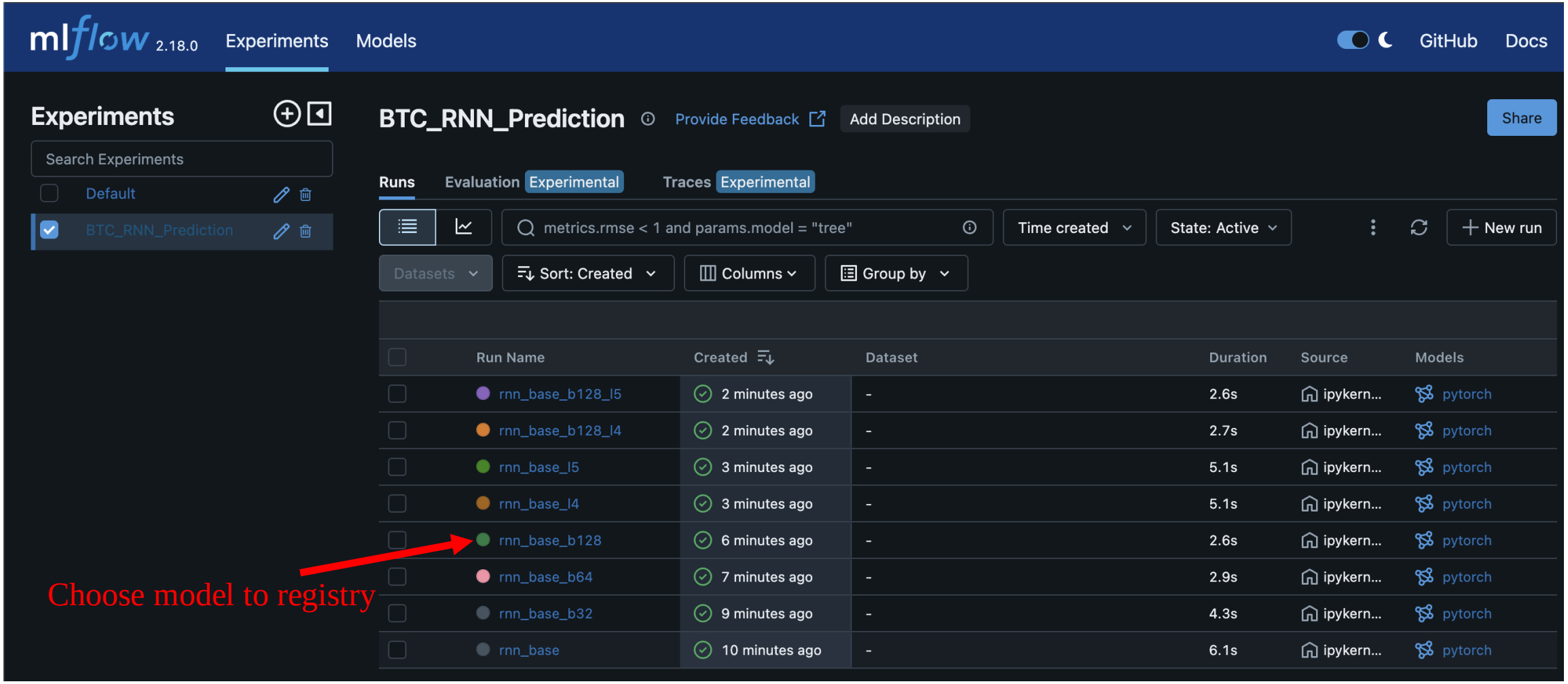
MLflow Registruy

❖ Model Registry Workflows



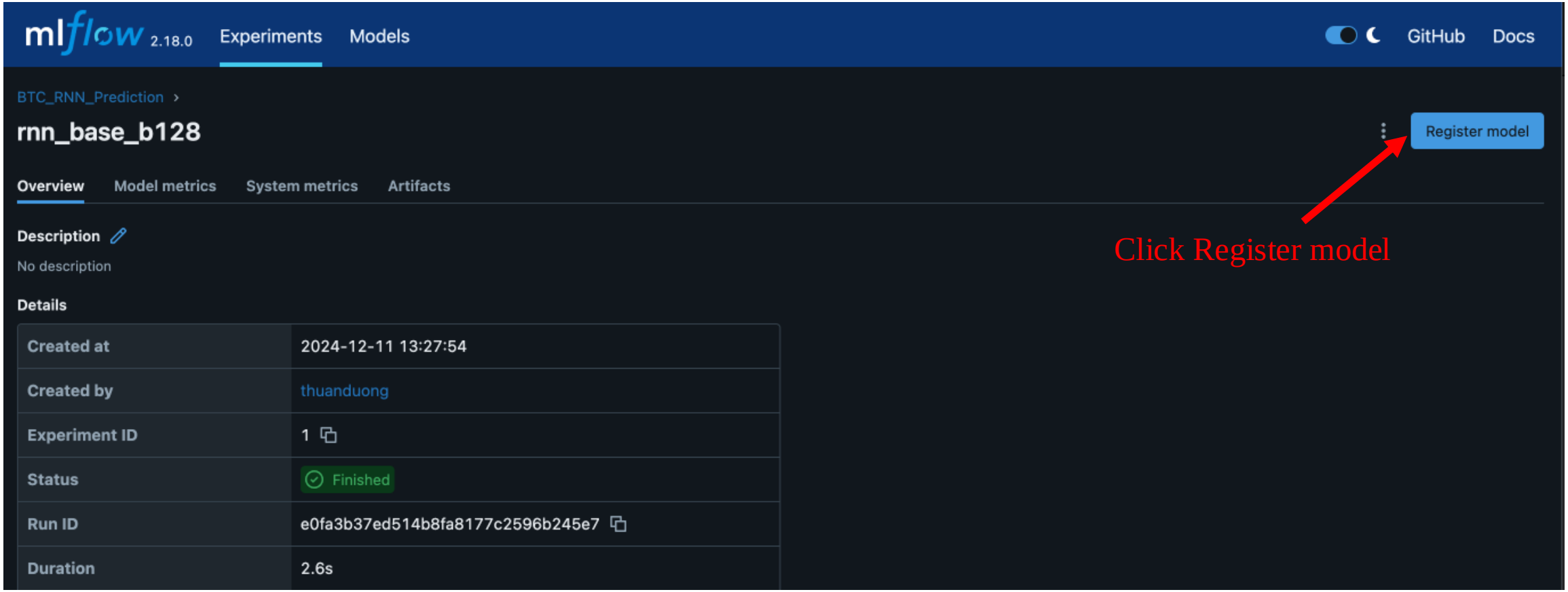
MLflow Registruy

❖ UI Workflow



MLflow Registruy

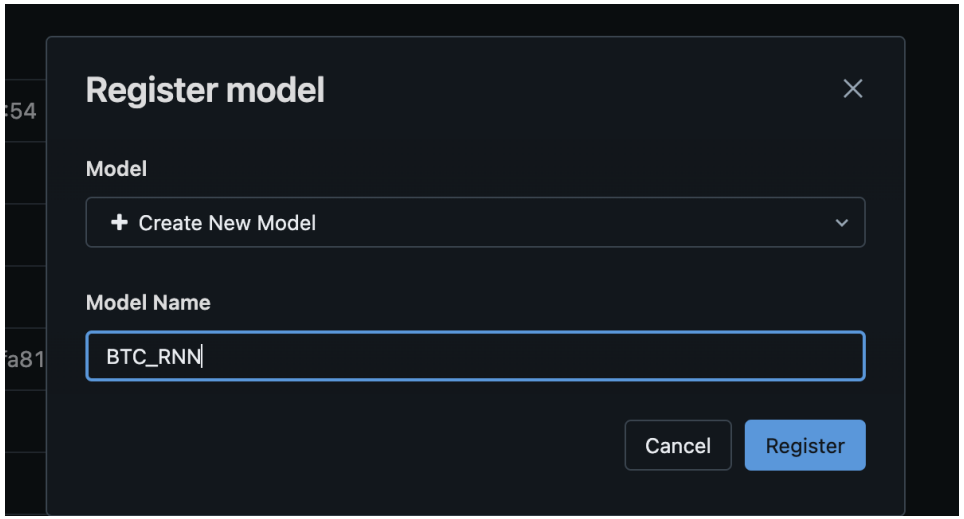
❖ UI Workflow

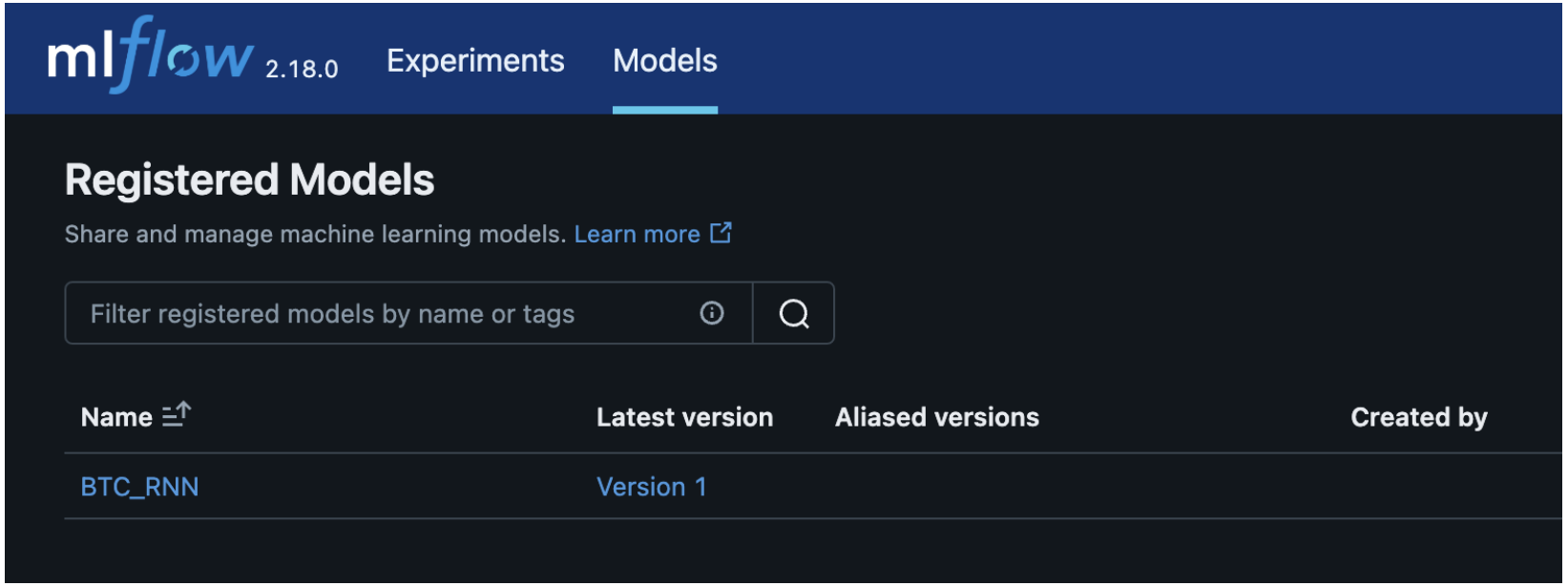


MLflow Registruy

❖ UI Workflow

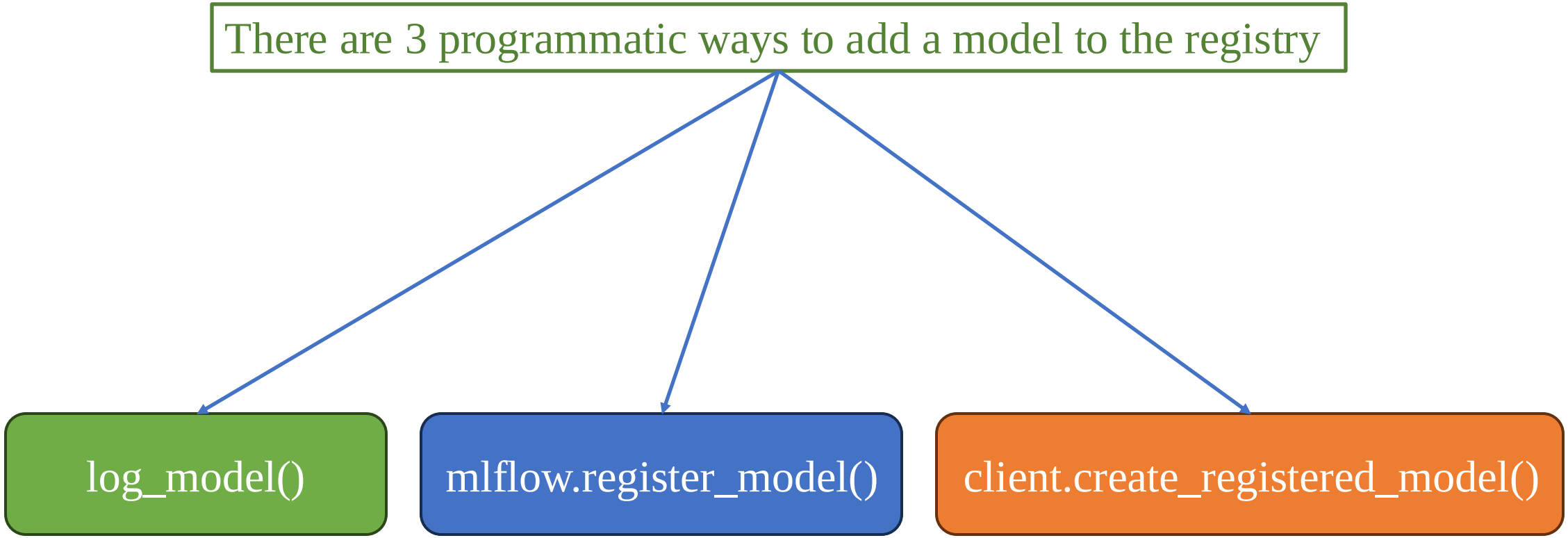
Create a name for the new model





MLflow Registry

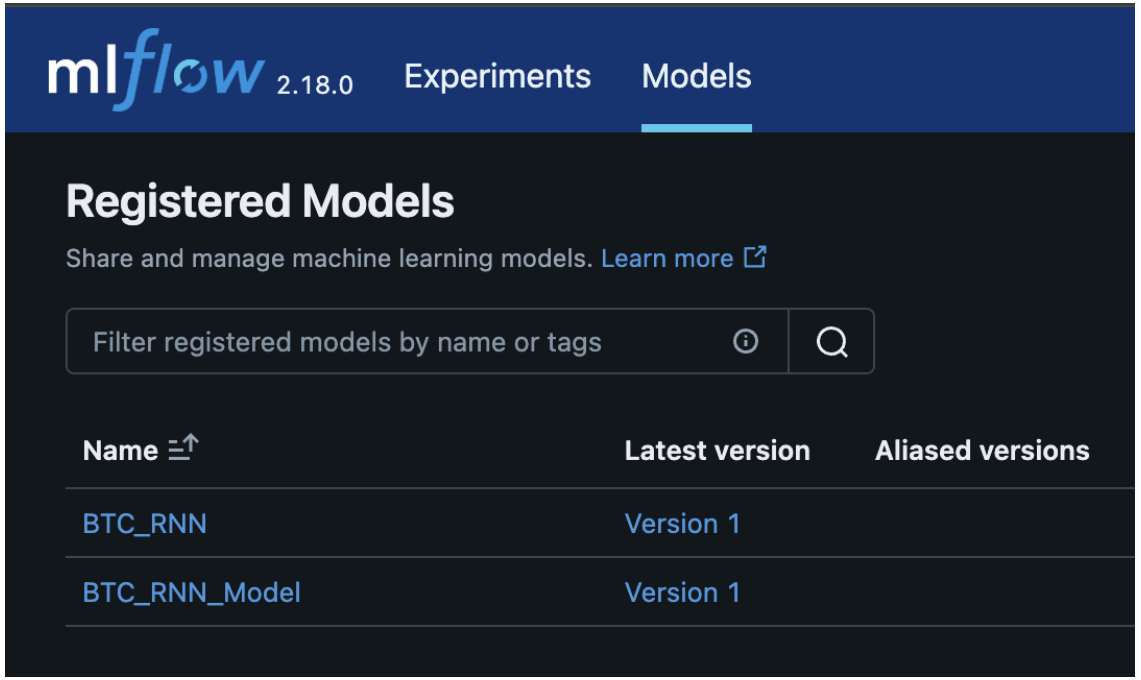
❖ API Workflows



MLflow Registruy

❖ Add when logging

```python
mlflow.pytorch.log_model(model,
                         artifact_path="model",
                         pip_requirements="./requirements.txt",
                         input_example=input_example,
                         registered_model_name="BTC_RNN_Model"
                         )
```



If the model name exists, it will increase the version

MLflow Registruy

❖ Use mlflow.register_model()

```
    model.load_state_dict(best_model_state_dict)
    input_example = torch.rand(1, LOOKBACK, input_size).numpy()
    mlflow.pytorch.log_model(model,
                             artifact_path="model",
                             pip_requirements="./requirements.txt",
                             input_example=input_example
                             )
mlflow.end_run()
```

[245] ✓ 2.0s

run_id

```
🏃 View run rnn_base_b128_l5 at: http://localhost:5000/#/experiments/1/runs/5ff6a9f3682a407e80c697a25f09646b
🧪 View experiment at: http://localhost:5000/#/experiments/1
```

```
result = mlflow.register_model(
    "runs:/5ff6a9f3682a407e80c697a25f09646b/pytorch", "BTC_RNN_via_MLflow"
)
```

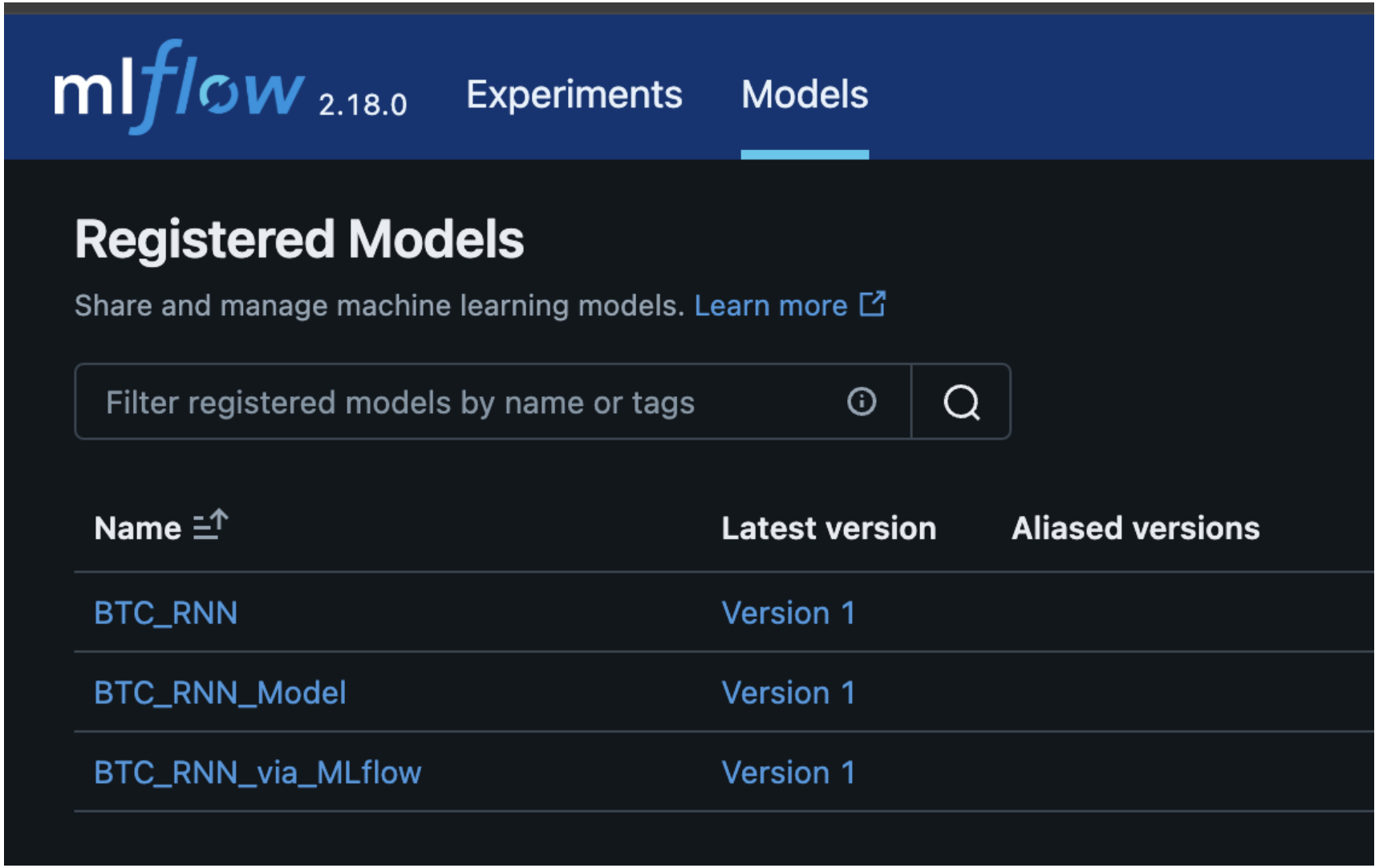
model name

[247] ✓ 0.0s

```
Registered model 'BTC_RNN_via_MLflow' already exists. Creating a new version of this model...
2024/12/11 13:59:41 INFO mlflow.store.model_registry.abstract_store: Waiting up to 300 seconds for model versi
Created version '1' of model 'BTC_RNN_via_MLflow'.
```

MLflow Registruy

❖ **Use mlflow.register_model()**



MLflow Registruy

❖ Use MLflow client

```python
from mlflow import MlflowClient

client = MlflowClient()
client.create_registered_model("BTC_RNN_via_client")
```

Create a empty registry model (None version)

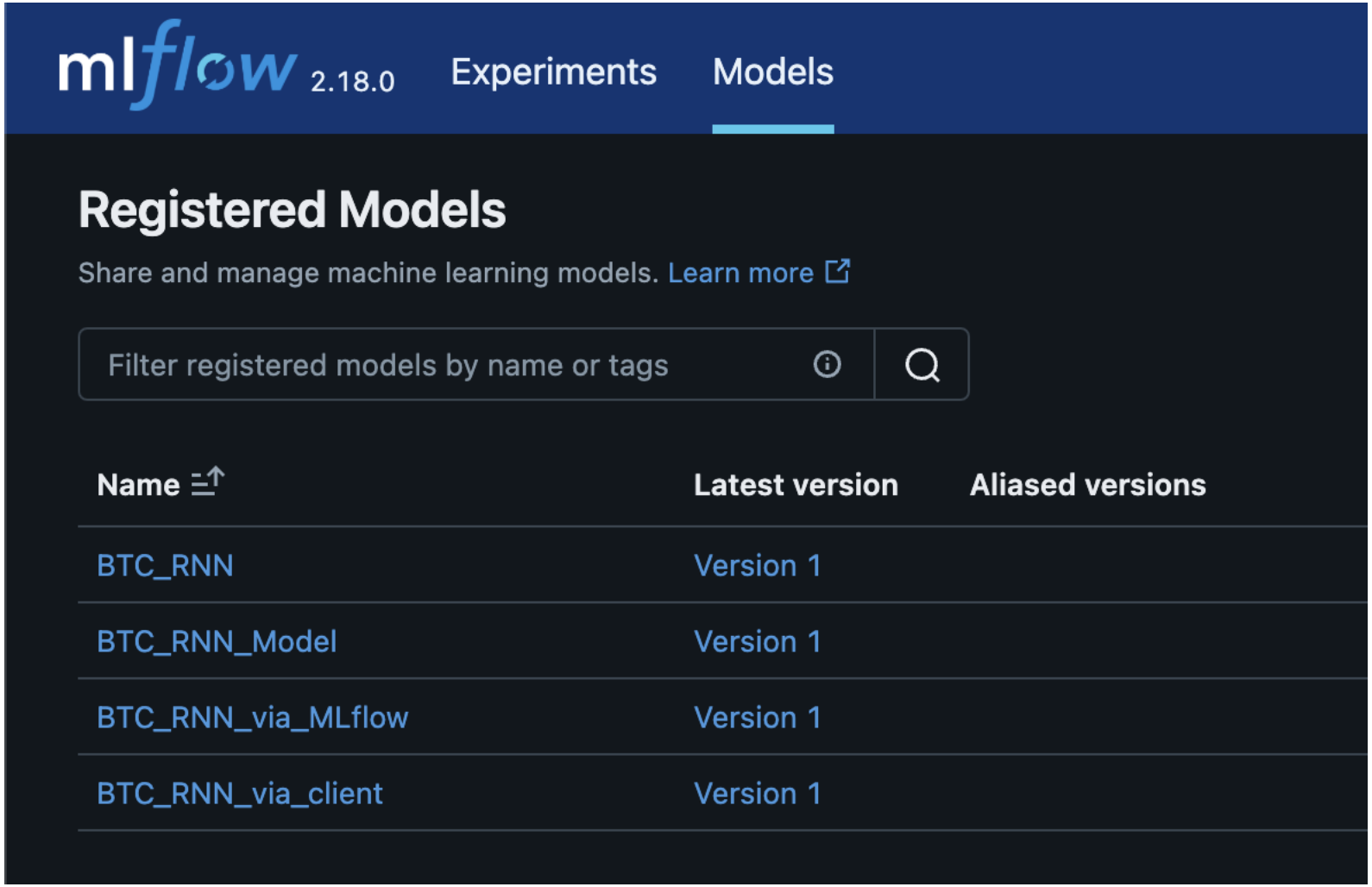
Create model version and artifacts

```python
client = MlflowClient()

result = client.create_model_version(
    name="BTC_RNN_via_client",
    source="mlflow-artifacts:/1/3c4fd76740164507a382ffcbe60f93ad/artifacts/model",
    run_id="3c4fd76740164507a382ffcbe60f93ad",
)
```

MLflow Registruy

❖ Use MLflow client

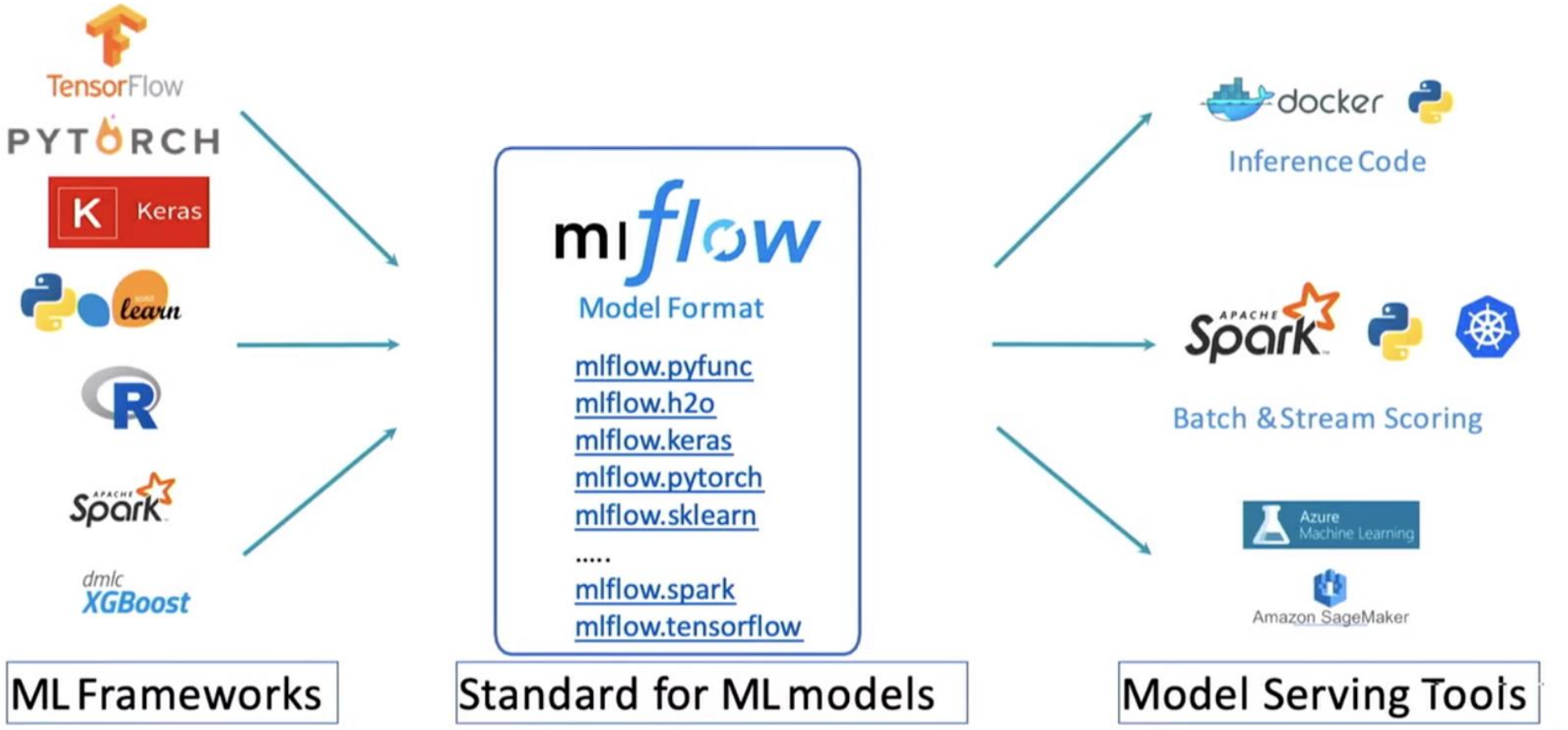


MLflow Model

MLflow Model

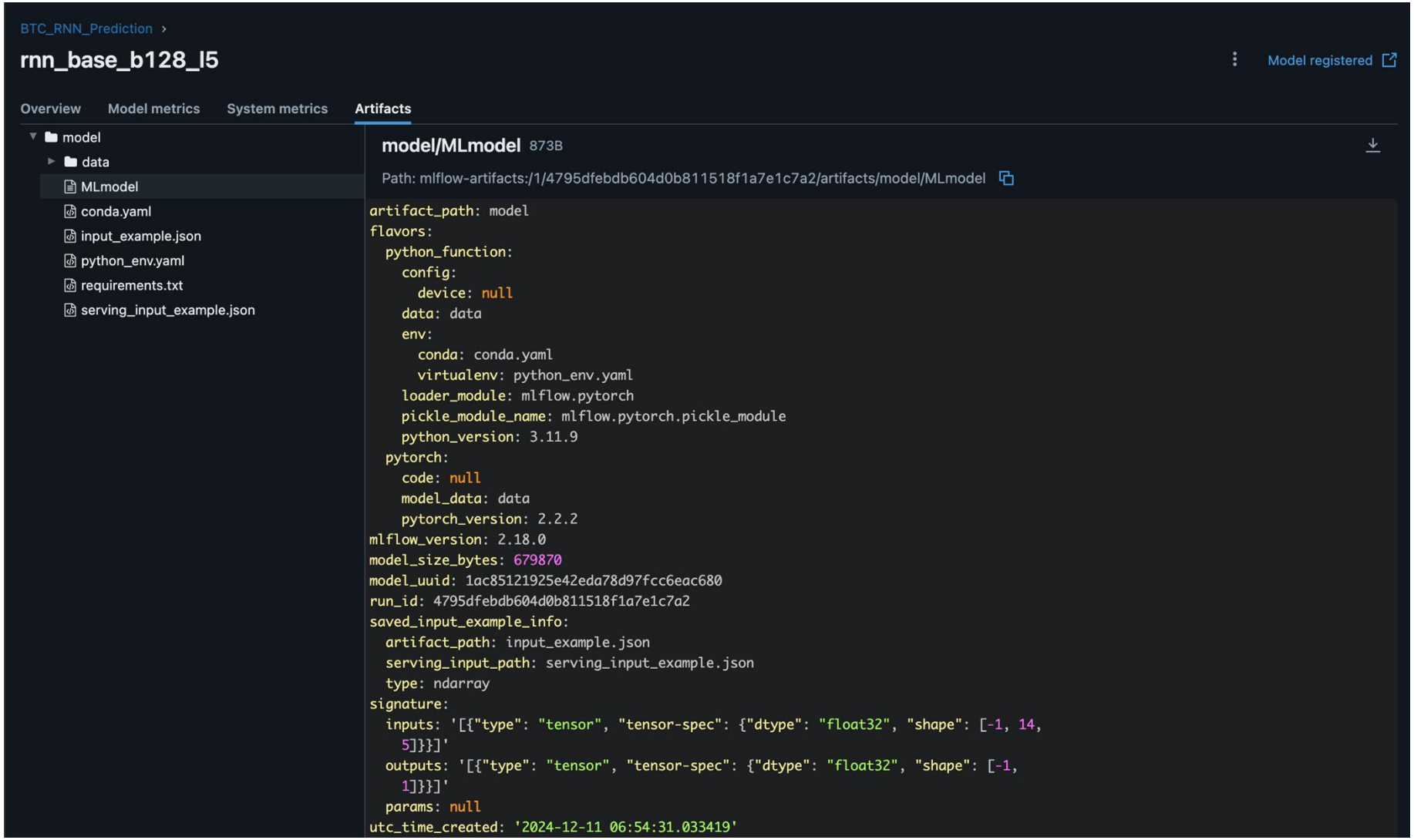
❖ Getting Started

- MLflow Model: packaging machine learning models so they can be deployed to a wide range of environments such as REST APIs, batch processing jobs, or cloud service.



MLflow Model

❖ MLmodel file



MLflow Model

❖ Model API

- Train and log the model with more framework options (e.g., Pytorch, Tensorflow, Sklearn, etc.)

```python
import mlflow
import mlflow.sklearn
from sklearn.ensemble import RandomForestClassifier

X = ...
y = ...
model = RandomForestClassifier().fit(X, y)

mlflow.sklearn.log_model(model, "rf_model")
```

```bash
#!/bin/bash

mlflow models serve -m models:rf_model/v1 --host 0.0.0.0 --port 5000

curl -X POST -H "Content-Type: application/json" --data '{"columns":["feature1", "feature2"], "data":[[1.0, 2.0]]}' http://127.0.0.1:5000/predict

```

MLflow Model

❖ Load model

```python
model_uri = 'runs:/3c4fd76740164507a382ffcbe60f93ad/model'

my_model = mlflow.pyfunc.load_model(model_uri)

type(my_model)
# >> mlflow.pyfunc.PyFuncModel
```

Load with `pyfunc`

Load to `pytorch` model

```python
model_uri = 'runs:/3c4fd76740164507a382ffcbe60f93ad/model'

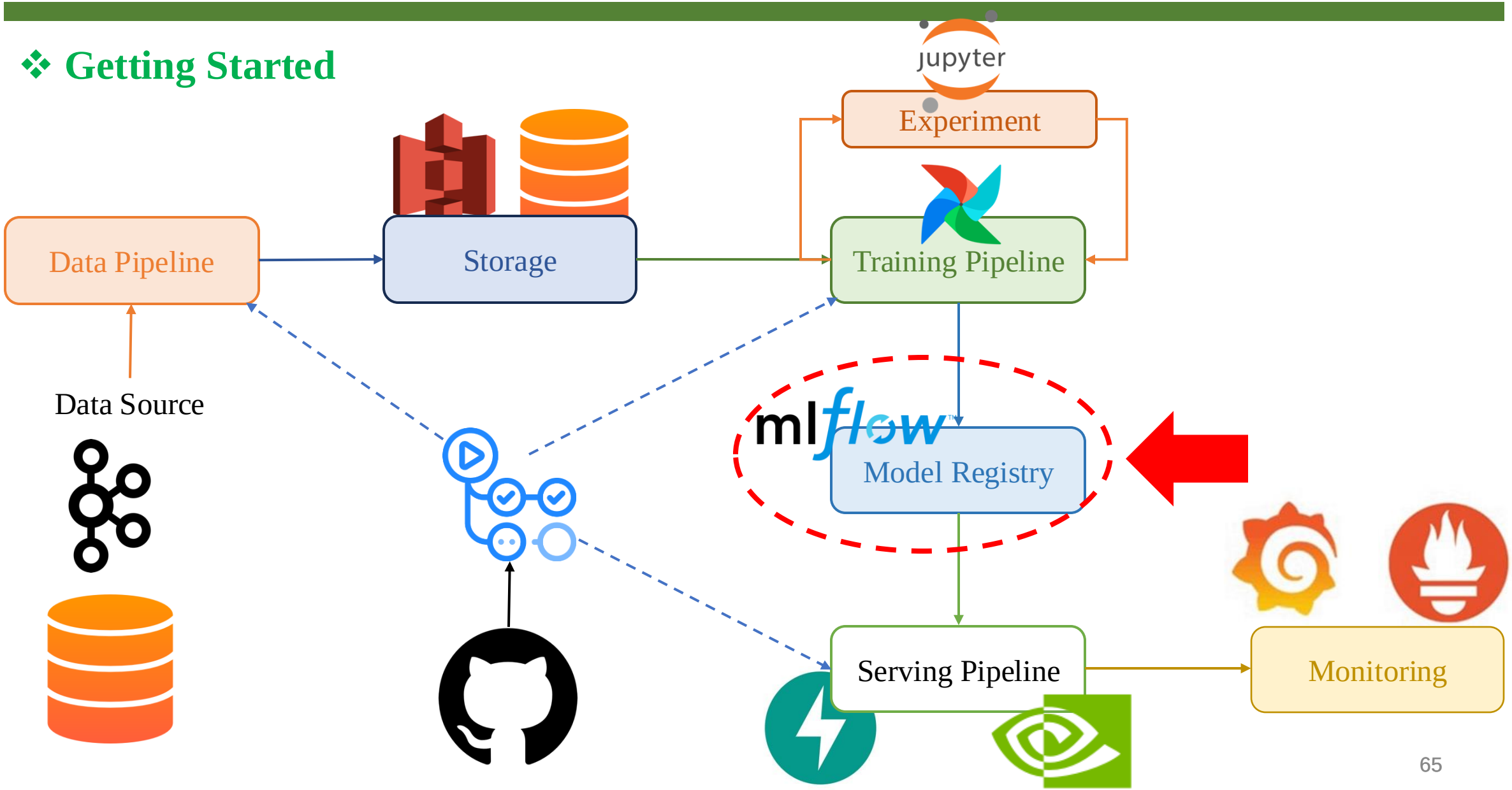
my_model = mlflow.pytorch.load_model(model_uri)

type(my_model)
# >> __main__.RNN_Model
```

Pipeline

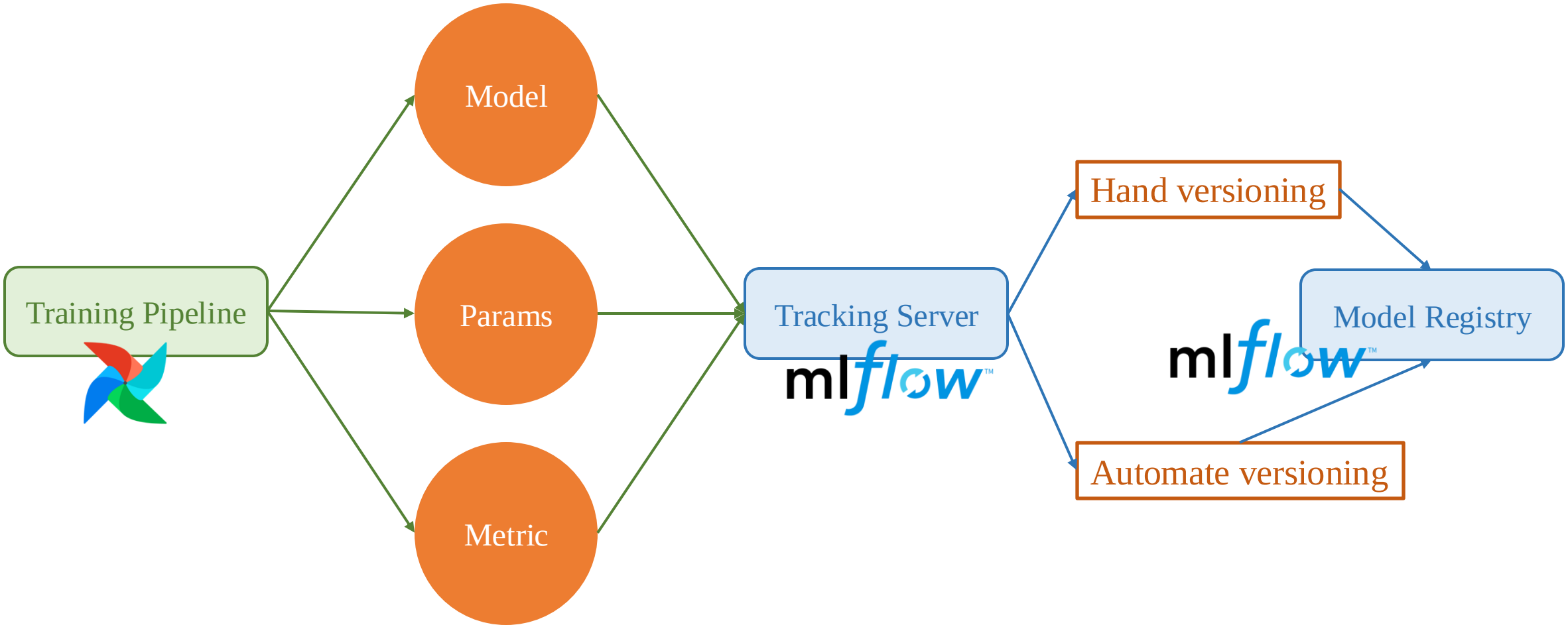
Pipeline

❖ Getting Started



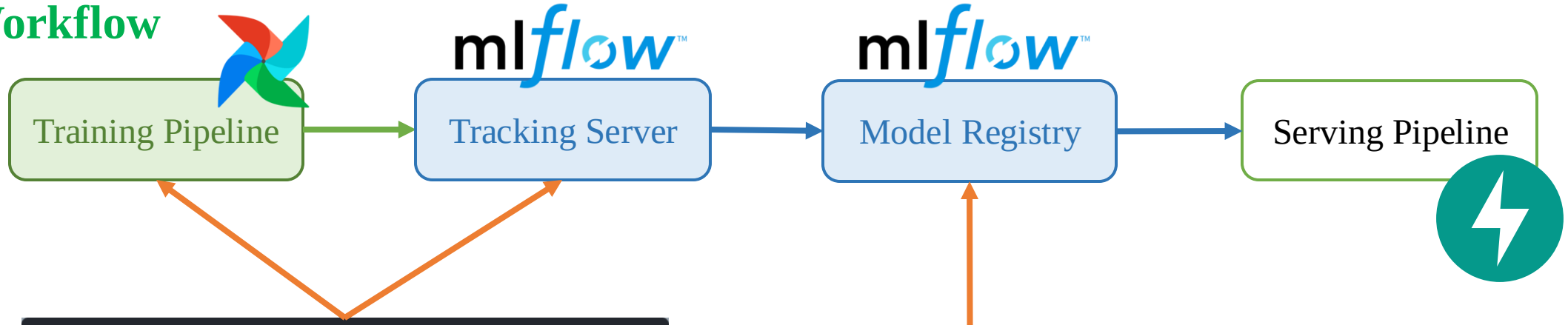
Pipeline

❖ Workflow



Pipeline

❖ Workflow




```python
data_processing_task = PythonOperator(
    task_id='data_processing',
    python_callable=data_processing,
    dag=dag,
)

train_model_task = PythonOperator(
    task_id='train_model',
    python_callable=train_model,
    dag=dag,
)

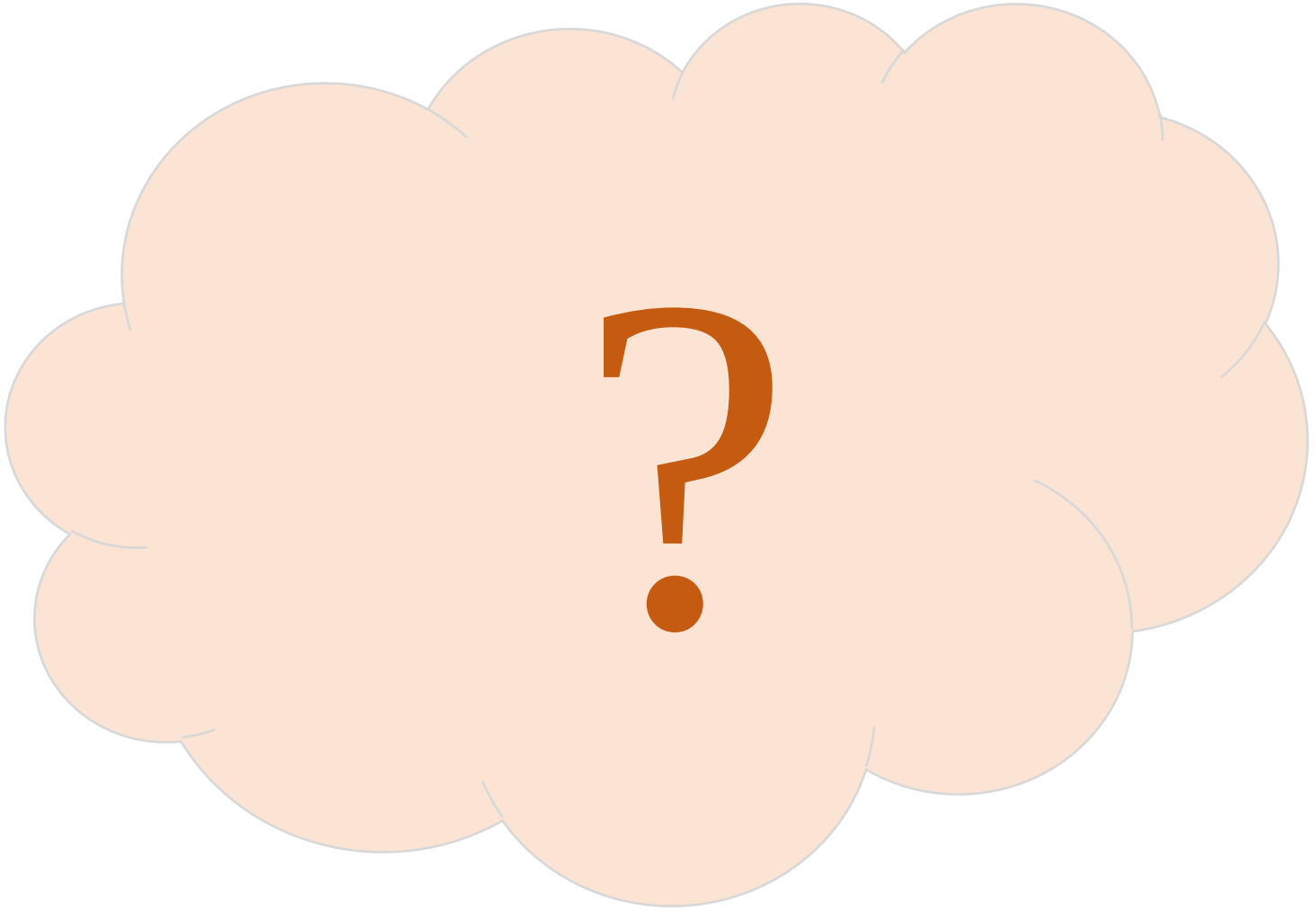
data_processing_task >> train_model_task
```

```python
validate_model_task = PythonOperator(
    task_id='validate_model',
    python_callable=validate_model,
    dag=dag,
)

validate_model_task
```

Practice

Question




Thank you!